

Lab10

Adrián González, Jose Ordoñez, Daniela Ramirez

2026-05-15

Link de repositorio en Git: <https://github.com/Ikeel04/Lab10-MD.git> (<https://github.com/Ikeel04/Lab10-MD.git>)

Librerías

```
library(dplyr)
```

```
##  
## Adjuntando el paquete: 'dplyr'
```

```
## The following objects are masked from 'package:stats':  
##  
##   filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(ggplot2)  
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
library(knitr)  
library(kableExtra)
```

```
## Warning: package 'kableExtra' was built under R version 4.5.3
```

```
##  
## Adjuntando el paquete: 'kableExtra'
```

```
## The following object is masked from 'package:dplyr':  
##  
##   group_rows
```

```
library(tidyr)  
library(scales)  
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.5.3
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##  
## Adjuntando el paquete: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':  
##  
##     margin
```

```
## The following object is masked from 'package:dplyr':  
##  
##     combine
```

```
library(dplyr)  
library(FNN)  
library(caret)
```

```
## Cargando paquete requerido: lattice
```

Selección del Conjunto de Datos

Origen y Justificación

Se utilizó el dataset **Heart Disease** disponible públicamente en Kaggle (<https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset> (<https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>)), el cual fue originalmente recopilado por el Cleveland Heart Disease Institute y forma parte del repositorio UCI Machine Learning Repository.

Este dataset resulta especialmente apropiado para aprendizaje semi-supervisado porque representa un escenario médico real donde obtener diagnósticos etiquetados es costoso y requiere personal especializado, mientras que datos clínicos básicos (no etiquetados) son fáciles de recopilar. Cuenta con 1,025 filas y 14 columnas (13 predictoras + 1 variable respuesta), cumpliendo los requisitos mínimos del laboratorio. Se puede ver que la variable objetivo es binaria y las clases están razonablemente balanceadas, lo que facilita la evaluación de métricas.

Carga del Dataset

```
heart <- read.csv("heart.csv")

cat("Dimensiones:", nrow(heart), "filas x", ncol(heart), "columnas\n")
```

```
## Dimensiones: 1025 filas x 14 columnas
```

Análisis Exploratorio de Datos

Estructura General

```
str(heart)
```

```
## 'data.frame': 1025 obs. of 14 variables:
## $ age : int 52 53 70 61 62 58 58 55 46 54 ...
## $ sex : int 1 1 1 1 0 0 1 1 1 1 ...
## $ cp : int 0 0 0 0 0 0 0 0 0 0 ...
## $ trestbps: int 125 140 145 148 138 100 114 160 120 122 ...
## $ chol : int 212 203 174 203 294 248 318 289 249 286 ...
## $ fbs : int 0 1 0 0 1 0 0 0 0 0 ...
## $ restecg : int 1 0 1 1 1 0 2 0 0 0 ...
## $ thalach : int 168 155 125 161 106 122 140 145 144 116 ...
## $ exang : int 0 1 1 0 0 0 0 1 0 1 ...
## $ oldpeak : num 1 3.1 2.6 0 1.9 1 4.4 0.8 0.8 3.2 ...
## $ slope : int 2 0 0 2 1 1 0 1 2 1 ...
## $ ca : int 2 0 0 1 3 0 3 1 0 2 ...
## $ thal : int 3 3 3 3 2 2 1 3 3 2 ...
## $ target : int 0 0 0 0 0 1 0 0 0 0 ...
```

```
summary(heart)
```

```

##      age      sex      cp      trestbps
##  Min.   :29.00  Min.   :0.0000  Min.   :0.0000  Min.   : 94.0
##  1st Qu.:48.00  1st Qu.:0.0000  1st Qu.:0.0000  1st Qu.:120.0
##  Median :56.00  Median :1.0000  Median :1.0000  Median :130.0
##  Mean   :54.43  Mean   :0.6956  Mean   :0.9424  Mean   :131.6
##  3rd Qu.:61.00  3rd Qu.:1.0000  3rd Qu.:2.0000  3rd Qu.:140.0
##  Max.   :77.00  Max.   :1.0000  Max.   :3.0000  Max.   :200.0
##      chol      fbs      restecg      thalach
##  Min.   :126  Min.   :0.0000  Min.   :0.0000  Min.   : 71.0
##  1st Qu.:211  1st Qu.:0.0000  1st Qu.:0.0000  1st Qu.:132.0
##  Median :240  Median :0.0000  Median :1.0000  Median :152.0
##  Mean   :246  Mean   :0.1493  Mean   :0.5298  Mean   :149.1
##  3rd Qu.:275  3rd Qu.:0.0000  3rd Qu.:1.0000  3rd Qu.:166.0
##  Max.   :564  Max.   :1.0000  Max.   :2.0000  Max.   :202.0
##      exang      oldpeak      slope      ca
##  Min.   :0.0000  Min.   :0.000  Min.   :0.000  Min.   :0.0000
##  1st Qu.:0.0000  1st Qu.:0.000  1st Qu.:1.000  1st Qu.:0.0000
##  Median :0.0000  Median :0.800  Median :1.000  Median :0.0000
##  Mean   :0.3366  Mean   :1.072  Mean   :1.385  Mean   :0.7541
##  3rd Qu.:1.0000  3rd Qu.:1.800  3rd Qu.:2.000  3rd Qu.:1.0000
##  Max.   :1.0000  Max.   :6.200  Max.   :2.000  Max.   :4.0000
##      thal      target
##  Min.   :0.000  Min.   :0.0000
##  1st Qu.:2.000  1st Qu.:0.0000
##  Median :2.000  Median :1.0000
##  Mean   :2.324  Mean   :0.5132
##  3rd Qu.:3.000  3rd Qu.:1.0000
##  Max.   :3.000  Max.   :1.0000

```

El dataset contiene **1025 observaciones** y **14 variables**, todas numéricas. La variable respuesta es `target` (0 = sin enfermedad cardíaca, 1 = con enfermedad cardíaca).

Descripción de Variables

Diccionario de variables – Heart Disease Dataset

Variable	Tipo	Descripción
age	Continua	Edad del paciente (años)
sex	Binaria	Sexo (1 = masculino, 0 = femenino)
cp	Categórica (0-3)	Tipo de dolor en el pecho (0=asintomático, 1=angina atípica, 2=no anginoso, 3=angina típica)
trestbps	Continua	Presión arterial en reposo (mmHg)
chol	Continua	Colesterol sérico (mg/dl)
fbs	Binaria	Azúcar en sangre en ayunas > 120 mg/dl (1 = verdadero)
restecg	Categórica (0-2)	Resultados del ECG en reposo (0=normal, 1=anormalidad ST-T, 2=hipertrofia VI)

Variable	Tipo	Descripción
thalach	Continua	Frecuencia cardíaca máxima alcanzada (bpm)
exang	Binaria	Angina inducida por ejercicio (1 = sí)
oldpeak	Continua	Depresión del segmento ST inducida por ejercicio
slope	Categorica (0-2)	Pendiente del segmento ST en ejercicio pico (0=descendente, 1=plana, 2=ascendente)
ca	Ordinal (0-4)	Número de vasos principales coloreados por fluoroscopia (0-4)
thal	Categorica (0-3)	Tipo de talasemia (0=normal, 1=defecto fijo, 2=defecto reversible, 3=desconocido)
target	Binaria	Diagnóstico de enfermedad cardíaca (0 = ausente, 1 = presente)

Valores Faltantes

```
nulos <- colSums(is.na(heart))
cat("Total de valores faltantes por columna:\n")
```

```
## Total de valores faltantes por columna:
```

```
print(nulos)
```

```
##      age      sex      cp trestbps      chol      fbs  restecg  thalach
##         0         0         0         0         0         0         0         0
##    exang oldpeak    slope      ca      thal    target
##         0         0         0         0         0         0
```

```
cat("\nTotal:", sum(nulos), "→ El dataset no requiere imputación.\n")
```

```
##
## Total: 0 → El dataset no requiere imputación.
```

Balance de Clases

```
dist_target <- heart %>%
  count(target) %>%
  mutate(
    Clase      = ifelse(target == 1, "Con enfermedad (1)", "Sin enfermedad (0)"),
    Porcentaje = round(n / sum(n) * 100, 1)
  )

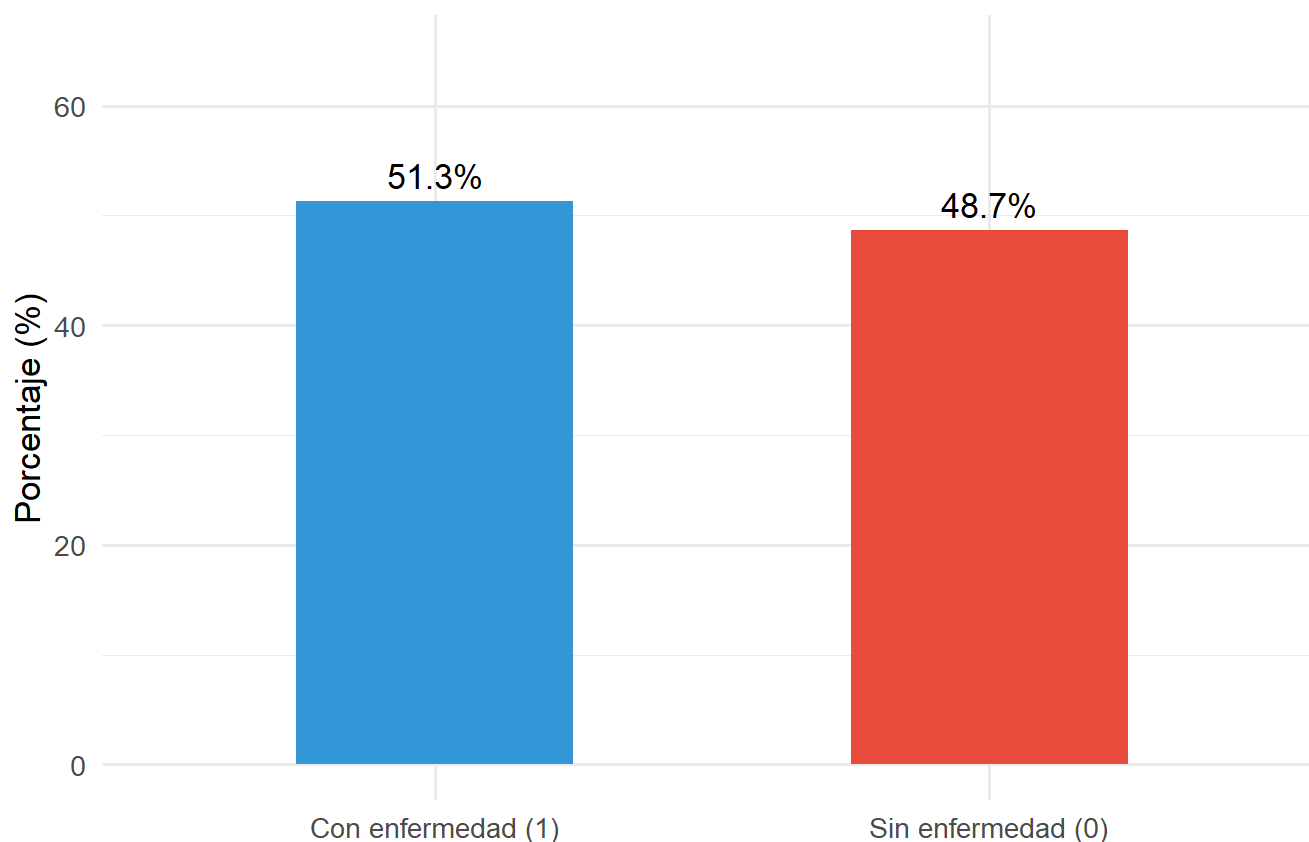
kable(dist_target[, c("Clase", "n", "Porcentaje")],
      col.names = c("Clase", "N", "%"),
      caption = "Distribución de la variable respuesta") %>%
kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Distribución de la variable
respuesta

Clase	N	%
Sin enfermedad (0)	499	48.7
Con enfermedad (1)	526	51.3

```
ggplot(dist_target, aes(x = Clase, y = Porcentaje, fill = Clase)) +
  geom_col(width = 0.5) +
  geom_text(aes(label = paste0(Porcentaje, "%")), vjust = -0.5, size = 4.5) +
  scale_fill_manual(values = c("#3498db", "#e74c3c")) +
  labs(title = "Balance de clases - Heart Disease",
       x = "", y = "Porcentaje (%)") +
  ylim(0, 65) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none")
```

Balance de clases – Heart Disease



Distribución de clases en la variable respuesta

Las clases están razonablemente balanceadas, lo que es favorable para el aprendizaje semi-supervisado: el modelo tiene una referencia equilibrada incluso con pocos datos etiquetados.

Estadísticas Descriptivas por Variable Continua

```
continuas <- c("age", "trestbps", "chol", "thalach", "oldpeak")

stats <- heart %>%
  select(all_of(continuas)) %>%
  summarise(across(everything(), list(
    Media = ~round(mean(.), 2),
    Mediana= ~round(median(.), 2),
    DE = ~round(sd(.), 2),
    Min = ~min(.),
    Max = ~max(.)
  ))) %>%
  pivot_longer(everything(), names_to = c("Variable", "Estadístico"),
    names_sep = "_") %>%
  pivot_wider(names_from = Estadístico, values_from = value)

kable(stats, caption = "Estadísticas descriptivas - variables continuas") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

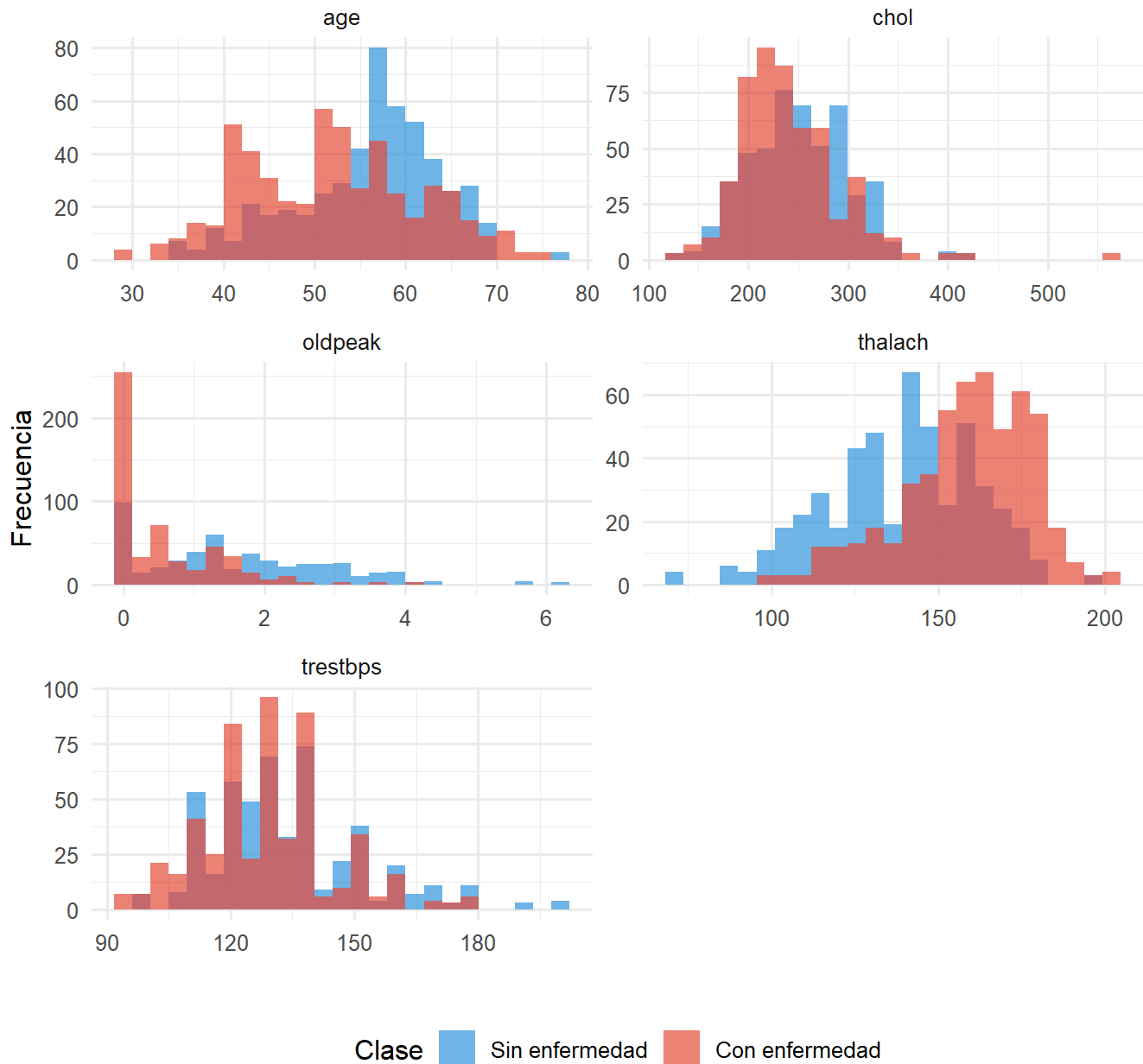
Estadísticas descriptivas – variables continuas

Variable	Media	Mediana	DE	Min	Max
age	54.43	56.0	9.07	29	77.0
trestbps	131.61	130.0	17.52	94	200.0
chol	246.00	240.0	51.59	126	564.0
thalach	149.11	152.0	23.01	71	202.0
oldpeak	1.07	0.8	1.18	0	6.2

Distribución de Variables Continuas

```
heart %>%
  select(all_of(continuas), target) %>%
  mutate(target = factor(target, labels = c("Sin enfermedad", "Con enfermedad"))) %>%
  pivot_longer(-target, names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = Valor, fill = target)) +
  geom_histogram(bins = 25, alpha = 0.7, position = "identity") +
  facet_wrap(~ Variable, scales = "free", ncol = 2) +
  scale_fill_manual(values = c("#3498db", "#e74c3c")) +
  labs(title = "Distribución de variables continuas por clase",
       x = "", y = "Frecuencia", fill = "Clase") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
```


Distribución de variables continuas por clase



Distribución de variables continuas

Se observa que la frecuencia cardíaca máxima (`thalach`) y la depresión del segmento ST (`oldpeak`) presentan las diferencias más marcadas entre pacientes con y sin enfermedad cardíaca, lo que sugiere alto poder discriminativo para los modelos.

Distribución de Variables Categóricas

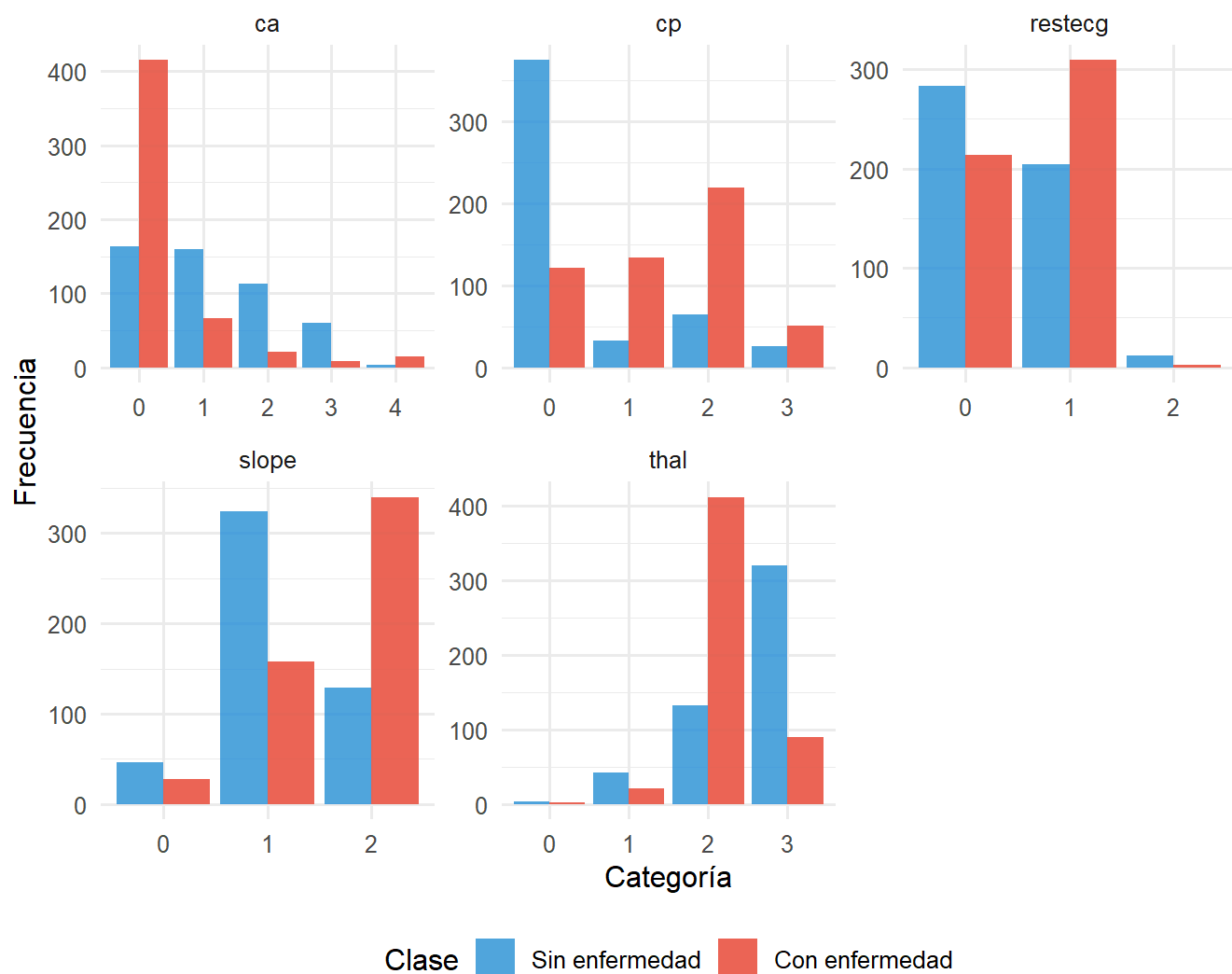
```

categoricas <- c("cp", "restecg", "slope", "ca", "thal")

heart %>%
  select(all_of(categoricas), target) %>%
  mutate(target = factor(target, labels = c("Sin enfermedad", "Con enfermedad"))) %>%
  pivot_longer(-target, names_to = "Variable", values_to = "Valor") %>%
  mutate(Valor = as.factor(Valor)) %>%
  ggplot(aes(x = Valor, fill = target)) +
  geom_bar(position = "dodge", alpha = 0.85) +
  facet_wrap(~ Variable, scales = "free", ncol = 3) +
  scale_fill_manual(values = c("#3498db", "#e74c3c")) +
  labs(title = "Variables categóricas por clase de diagnóstico",
       x = "Categoría", y = "Frecuencia", fill = "Clase") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

```

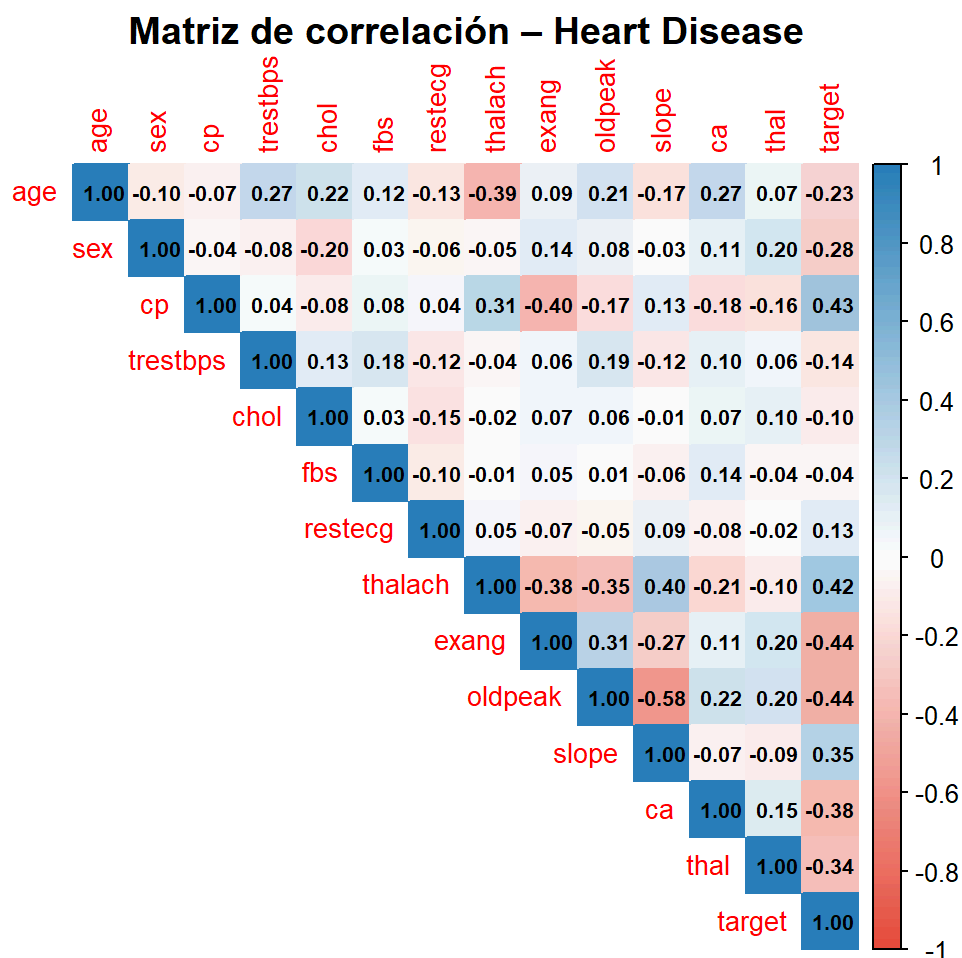
Variables categóricas por clase de diagnóstico



Variables categóricas por clase

Análisis de Correlación

```
M <- cor(heart)
corrplot(M,
  method = "color",
  type = "upper",
  tl.cex = 0.85,
  addCoef.col = "black",
  number.cex = 0.65,
  col = colorRampPalette(c("#e74c3c", "white", "#2980b9"))(200),
  title = "Matriz de correlación - Heart Disease",
  mar = c(0, 0, 1, 0))
```



Mapa de correlación entre variables numéricas

```
cor_target <- sort(abs(cor(heart)[, "target"]), decreasing = TRUE)
cor_df <- data.frame(
  Variable = names(cor_target),
  Correlacion = round(cor(heart)[names(cor_target), "target"], 3)
)

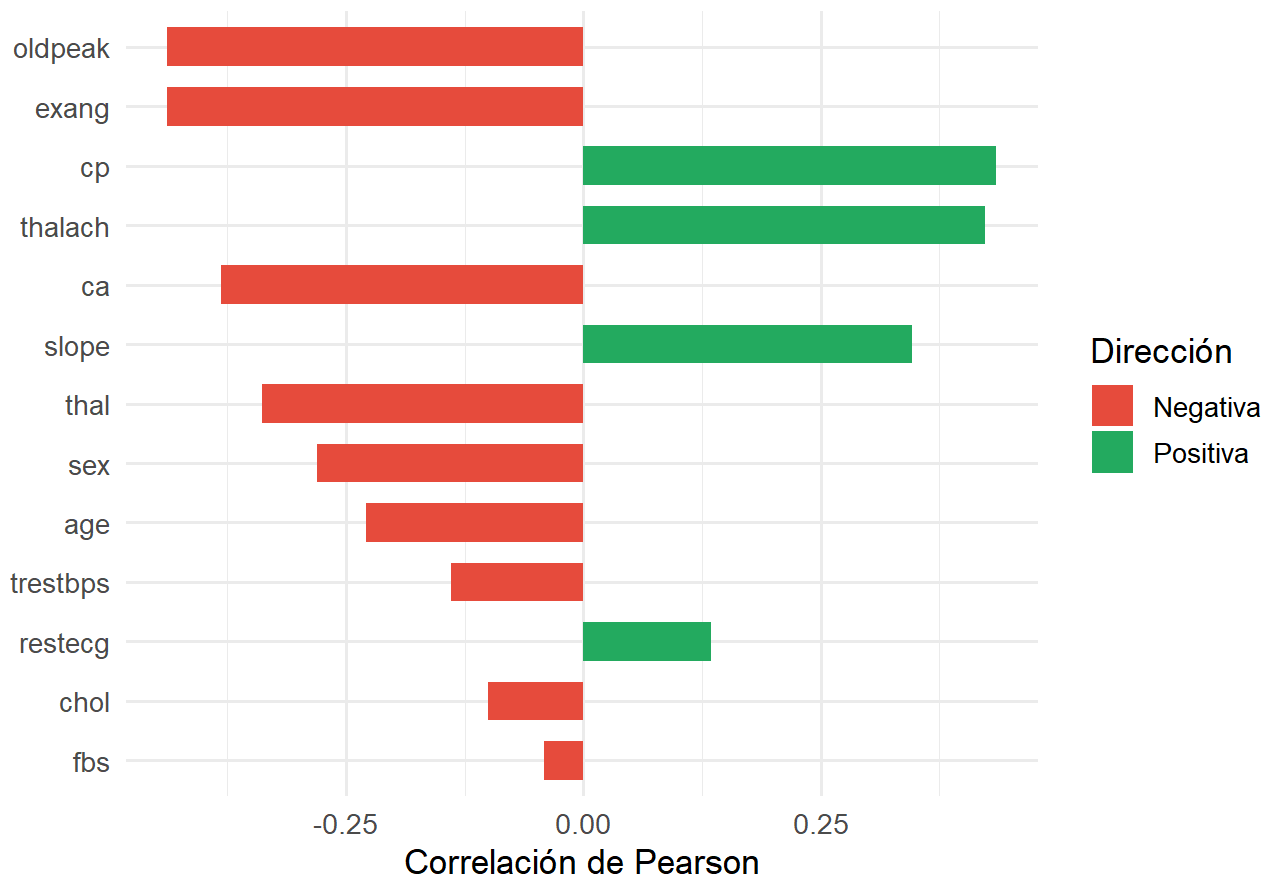
kable(cor_df[-1, ], row.names = FALSE,
  caption = "Correlación de cada variable con la variable respuesta (target)") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

Correlación de cada variable con la variable respuesta (target)

Variable	Correlacion
oldpeak	-0.438
exang	-0.438
cp	0.435
thalach	0.423
ca	-0.382
slope	0.346
thal	-0.338
sex	-0.280
age	-0.229
trestbps	-0.139
restecg	0.134
chol	-0.100
fbs	-0.041

```
ggplot(cor_df[-1, ], aes(x = reorder(Variable, abs(Correlacion)),
                          y = Correlacion,
                          fill = Correlacion > 0)) +
  geom_col(width = 0.65) +
  coord_flip() +
  scale_fill_manual(values = c("#e74c3c", "#27ae60"),
                    labels = c("Negativa", "Positiva")) +
  labs(title = "Correlación de predictoras con target",
       x = "", y = "Correlación de Pearson", fill = "Dirección") +
  theme_minimal(base_size = 13)
```

Correlación de predictoras con target



Correlación con la variable respuesta

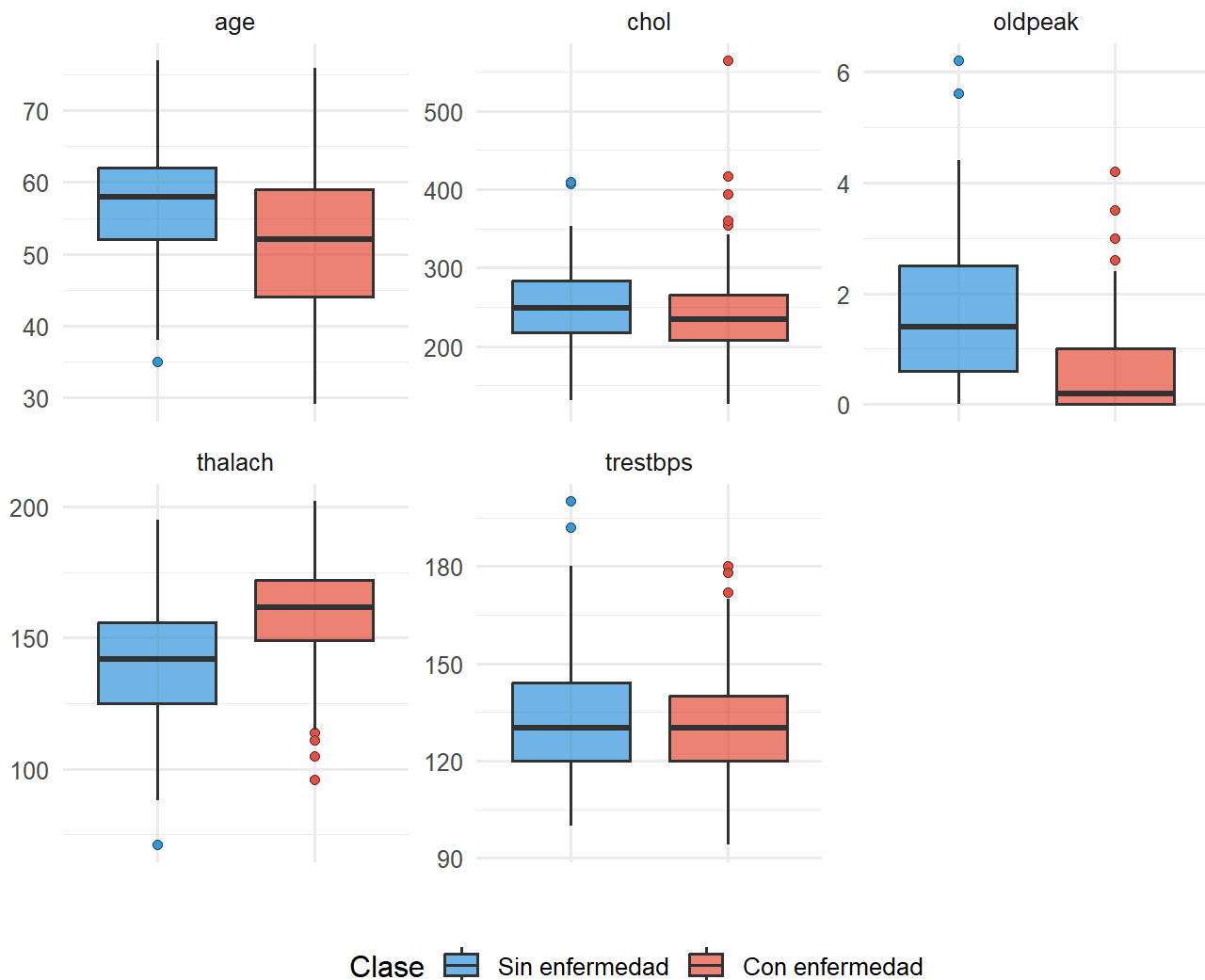
Las variables con mayor correlación absoluta con target son:

- **cp** (tipo de dolor en el pecho): correlación positiva, pacientes con angina típica tienen mayor probabilidad de enfermedad.
- **thalach** (frecuencia cardíaca máxima): correlación positiva, frecuencias más altas asociadas a presencia de enfermedad.
- **exang** (angina por ejercicio): correlación negativa, su presencia está asociada a diagnóstico positivo.
- **oldpeak** (depresión ST): correlación negativa, valores altos indican mayor riesgo.
- **ca** (vasos principales): correlación negativa marcada, más vasos obstruidos indican enfermedad.

Boxplots: Variables Continuas por Clase

```
heart %>%
  select(all_of(continuas), target) %>%
  mutate(target = factor(target, labels = c("Sin enfermedad", "Con enfermedad"))) %>%
  pivot_longer(-target, names_to = "Variable", values_to = "Valor") %>%
  ggplot(aes(x = target, y = Valor, fill = target)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 21) +
  facet_wrap(~ Variable, scales = "free_y", ncol = 3) +
  scale_fill_manual(values = c("#3498db", "#e74c3c")) +
  labs(title = "Variables continuas por clase de diagnóstico",
       x = "", y = "", fill = "Clase") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom",
       axis.text.x = element_blank())
```

Variables continuas por clase de diagnóstico



Distribución de variables continuas por diagnóstico

Preprocesamiento

Las variables categóricas (cp , restecg , slope , thal) serán tratadas como factores en los modelos que lo requieran. Para los algoritmos semi-supervisados se aplicará **estandarización** (media 0, desviación estándar 1) sobre todas las variables numéricas, utilizando exclusivamente los parámetros del conjunto de entrenamiento para evitar data leakage.

```
# Convertir variables categóricas a factor
cat_vars <- c("sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal", "target")
heart_factor <- heart %>%
  mutate(across(all_of(cat_vars), as.factor))

str(heart_factor)
```

```
## 'data.frame': 1025 obs. of 14 variables:
## $ age : int 52 53 70 61 62 58 58 55 46 54 ...
## $ sex : Factor w/ 2 levels "0","1": 2 2 2 2 1 1 2 2 2 2 ...
## $ cp : Factor w/ 4 levels "0","1","2","3": 1 1 1 1 1 1 1 1 1 1 ...
## $ trestbps: int 125 140 145 148 138 100 114 160 120 122 ...
## $ chol : int 212 203 174 203 294 248 318 289 249 286 ...
## $ fbs : Factor w/ 2 levels "0","1": 1 2 1 1 2 1 1 1 1 1 ...
## $ restecg : Factor w/ 3 levels "0","1","2": 2 1 2 2 2 1 3 1 1 1 ...
## $ thalach : int 168 155 125 161 106 122 140 145 144 116 ...
## $ exang : Factor w/ 2 levels "0","1": 1 2 2 1 1 1 1 2 1 2 ...
## $ oldpeak : num 1 3.1 2.6 0 1.9 1 4.4 0.8 0.8 3.2 ...
## $ slope : Factor w/ 3 levels "0","1","2": 3 1 1 3 2 2 1 2 3 2 ...
## $ ca : Factor w/ 5 levels "0","1","2","3",...: 3 1 1 2 4 1 4 2 1 3 ...
## $ thal : Factor w/ 4 levels "0","1","2","3": 4 4 4 4 3 3 2 4 4 3 ...
## $ target : Factor w/ 2 levels "0","1": 1 1 1 1 1 2 1 1 1 1 ...
```

```
cat("\nResumen de la variable respuesta tras preprocesamiento:\n")
```

```
##
## Resumen de la variable respuesta tras preprocesamiento:
```

```
print(table(heart_factor$target))
```

```
##
## 0 1
## 499 526
```

El dataset queda listo para el diseño experimental: sin valores faltantes, con variables categóricas correctamente tipificadas y con clases balanceadas. La estandarización de variables continuas se aplicará dentro de cada partición experimental para evitar contaminación entre conjuntos de entrenamiento y prueba.

Diseño experimental

```
set.seed(123)

heart$target <- as.factor(heart$target)

train_index <- createDataPartition(heart$target, p = 0.7, list = FALSE)

train_data <- heart[train_index, ]
test_data <- heart[-train_index, ]

train_data$target <- as.factor(train_data$target)
test_data$target <- as.factor(test_data$target)

dim(train_data)
```

```
## [1] 719 14
```

```
dim(test_data)
```

```
## [1] 306 14
```

El dataset fue dividido en un conjunto de entrenamiento (70%) y uno de prueba (30%) utilizando muestreo estratificado con `createDataPartition()`. Esto permite mantener una distribución similar de las clases en ambos conjuntos. Los datos de entrenamiento se utilizarán para construir los modelos, mientras que los datos de prueba servirán para evaluar su desempeño.

```
porcentajes_etiquetados <- c(0.05, 0.10, 0.20)
porcentajes_etiquetados
```

```
## [1] 0.05 0.10 0.20
```



```

set.seed(123)

crear_particion_semi <- function(data, porcentaje) {
  labeled_index <- createDataPartition(
    data$target,
    p = porcentaje,
    list = FALSE
  )

  labeled_data <- data[labeled_index, ]
  unlabeled_data <- data[-labeled_index, ]

  labeled_data$target <- as.factor(labeled_data$target)
  unlabeled_data$target <- NA

  list(
    labeled = labeled_data,
    unlabeled = unlabeled_data
  )
}

particion_5 <- crear_particion_semi(train_data, 0.05)
particion_10 <- crear_particion_semi(train_data, 0.10)
particion_20 <- crear_particion_semi(train_data, 0.20)

data.frame(
  porcentaje = c("5%", "10%", "20%"),
  etiquetados = c(
    nrow(particion_5$labeled),
    nrow(particion_10$labeled),
    nrow(particion_20$labeled)
  ),
  no_etiquetados = c(
    nrow(particion_5$unlabeled),
    nrow(particion_10$unlabeled),
    nrow(particion_20$unlabeled)
  )
)

```

```

##  porcentaje etiquetados no_etiquetados
## 1      5%          37          682
## 2     10%          72          647
## 3     20%         144          575

```

Para simular un escenario de aprendizaje semi supervisado, se utilizaron diferentes porcentajes de datos etiquetados dentro del conjunto de entrenamiento: 5%, 10% y 20%. El resto de las observaciones fueron consideradas como datos no etiquetados, reemplazando temporalmente la variable objetivo por valores NA. Esto permitirá comparar el desempeño de modelos supervisados y semi supervisados bajo distintas cantidades de información etiquetada.

```

evaluar_modelo <- function(predicciones, reales, nombre_modelo, porcentaje) {
  cm <- confusionMatrix(
    data = as.factor(predicciones),
    reference = as.factor(reales),
    positive = "1"
  )

  data.frame(
    modelo = nombre_modelo,
    porcentaje_etiquetado = porcentaje,
    accuracy = cm$overall["Accuracy"],
    precision = cm$byClass["Precision"],
    recall = cm$byClass["Recall"],
    f1 = cm$byClass["F1"]
  )
}

```

Esta función se utiliza para evaluar el desempeño de los modelos implementados. A partir de las predicciones generadas y las etiquetas reales del conjunto de prueba, se calcula la matriz de confusión y se obtienen métricas de evaluación como accuracy, precision, recall y F1-score. Estas métricas permiten comparar objetivamente el rendimiento de los modelos supervisados y semi supervisados.

```

entrenar_supervisado <- function(particion, test_data, porcentaje) {
  labeled_data <- particion$labeled
  labeled_data$target <- as.factor(labeled_data$target)

  modelo <- randomForest(
    target ~ .,
    data = labeled_data,
    ntree = 300
  )

  predicciones <- predict(modelo, newdata = test_data)

  evaluar_modelo(
    predicciones = predicciones,
    reales = test_data$target,
    nombre_modelo = "Supervisado - Random Forest",
    porcentaje = porcentaje
  )
}

```

Este modelo corresponde al enfoque supervisado tradicional utilizado como baseline. El algoritmo Random Forest se entrena únicamente con los datos etiquetados disponibles y posteriormente realiza predicciones sobre el conjunto de prueba. Los resultados obtenidos permiten comparar el desempeño del aprendizaje supervisado frente a los métodos semi supervisados implementados.

```
entrenar_self_training_v2 <- function(particion, test_data, porcentaje,
                                     threshold = 0.85, max_iter = 5) {

  labeled_data <- particion$labeled
  unlabeled_data <- particion$unlabeled

  labeled_data$target <- as.factor(labeled_data$target)
  niveles <- levels(labeled_data$target)

  for (i in 1:max_iter) {

    unlabeled_features <- unlabeled_data %>%
      select(-target)

    modelo <- randomForest(
      target ~ .,
      data = labeled_data,
      ntree = 300
    )

    probabilidades <- predict(
      modelo,
      newdata = unlabeled_features,
      type = "prob"
    )

    confianza <- apply(probabilidades, 1, max)
    pseudo_labels <- colnames(probabilidades)[apply(probabilidades, 1, which.max)]

    seleccionados <- which(confianza >= threshold)

    if (length(seleccionados) == 0) {
      break
    }

    nuevos_datos <- unlabeled_features[seleccionados, , drop = FALSE]

    nuevos_datos <- cbind(
      nuevos_datos,
      target = factor(pseudo_labels[seleccionados], levels = niveles)
    )

    labeled_data <- bind_rows(labeled_data, nuevos_datos)
    unlabeled_data <- unlabeled_data[-seleccionados, , drop = FALSE]

    if (nrow(unlabeled_data) == 0) {
      break
    }
  }

  modelo_final <- randomForest(
    target ~ .,
```

```
    data = labeled_data,  
    ntree = 300  
  )  
  
predicciones <- predict(modelo_final, newdata = test_data)  
  
evaluar_modelo(  
  predicciones = predicciones,  
  reales = test_data$target,  
  nombre_modelo = "Semi-supervisado - Self-Training RF",  
  porcentaje = porcentaje  
)  
}
```

Este modelo implementa Self-Training utilizando Random Forest. Inicialmente, el modelo se entrena únicamente con los datos etiquetados y posteriormente genera predicciones sobre los datos no etiquetados. Las observaciones con mayor nivel de confianza son seleccionadas y agregadas al conjunto etiquetado como pseudo-etiquetas. Este proceso se repite iterativamente para aprovechar la información de los datos sin etiqueta y mejorar el entrenamiento del modelo.

```
entrenar_label_propagation <- function(particion, test_data, porcentaje,
                                       k = 7, alpha = 0.8, iteraciones = 100) {

  labeled_data <- particion$labeled
  unlabeled_data <- particion$unlabeled

  labeled_data$target <- as.factor(labeled_data$target)
  niveles <- levels(labeled_data$target)

  semi_data <- bind_rows(labeled_data, unlabeled_data)

  indices_etiquetados <- 1:nrow(labeled_data)

  X <- semi_data %>%
    select(-target)

  X <- model.matrix(~ ., data = X)[, -1]
  X <- scale(X)

  knn <- get.knn(X, k = k)

  n <- nrow(X)
  W <- matrix(0, nrow = n, ncol = n)

  sigma <- median(knn$nn.dist)

  for (i in 1:n) {
    vecinos <- knn$nn.index[i, ]
    distancias <- knn$nn.dist[i, ]
    pesos <- exp(-(distancias^2) / (2 * sigma^2))
    W[i, vecinos] <- pesos
  }

  W <- W / rowSums(W)
  W[is.na(W)] <- 0

  Y <- matrix(0, nrow = n, ncol = length(niveles))
  colnames(Y) <- niveles

  for (i in indices_etiquetados) {
    Y[i, as.character(labeled_data$target[i])] <- 1
  }

  F <- Y

  for (i in 1:iteraciones) {
    F <- alpha * W %*% F + (1 - alpha) * Y
    F[indices_etiquetados, ] <- Y[indices_etiquetados, ]
  }

  pseudo_labels <- colnames(F)[max.col(F)]
```

```
semi_data_completo <- semi_data %>%
  select(-target)

semi_data_completo$target <- factor(
  pseudo_labels,
  levels = niveles
)

modelo_final <- randomForest(
  target ~ .,
  data = semi_data_completo,
  ntree = 300
)

predicciones <- predict(modelo_final, newdata = test_data)

evaluar_modelo(
  predicciones = predicciones,
  reales = test_data$target,
  nombre_modelo = "Semi-supervisado - Label Propagation",
  porcentaje = porcentaje
)
}
```

Este modelo utiliza Label Propagation, un método semi supervisado basado en grafos. Primero se combinan los datos etiquetados y no etiquetados, y luego se construye una red de similitud utilizando k-NN. Las etiquetas conocidas se propagan iterativamente hacia los datos sin etiqueta para generar pseudo-etiquetas, las cuales posteriormente son utilizadas para entrenar un modelo Random Forest.

```
resultados <- bind_rows(
  entrenar_supervisado(particion_5, test_data, "5%"),
  entrenar_supervisado(particion_10, test_data, "10%"),
  entrenar_supervisado(particion_20, test_data, "20%"),

  entrenar_self_training_v2(particion_5, test_data, "5%"),
  entrenar_self_training_v2(particion_10, test_data, "10%"),
  entrenar_self_training_v2(particion_20, test_data, "20%"),

  entrenar_label_propagation(particion_5, test_data, "5%"),
  entrenar_label_propagation(particion_10, test_data, "10%"),
  entrenar_label_propagation(particion_20, test_data, "20%")
)

resultados
```

```
##                                modelo porcentaje_etiquetado
## Accuracy...1      Supervisado - Random Forest           5%
## Accuracy...2      Supervisado - Random Forest          10%
## Accuracy...3      Supervisado - Random Forest          20%
## Accuracy...4 Semi-supervisado - Self-Training RF         5%
## Accuracy...5 Semi-supervisado - Self-Training RF        10%
## Accuracy...6 Semi-supervisado - Self-Training RF        20%
## Accuracy...7 Semi-supervisado - Label Propagation        5%
## Accuracy...8 Semi-supervisado - Label Propagation       10%
## Accuracy...9 Semi-supervisado - Label Propagation       20%
##          accuracy precision    recall      f1
## Accuracy...1 0.7875817 0.7987013 0.7834395 0.7909968
## Accuracy...2 0.8496732 0.8675497 0.8343949 0.8506494
## Accuracy...3 0.8790850 0.8797468 0.8853503 0.8825397
## Accuracy...4 0.7614379 0.7658228 0.7707006 0.7682540
## Accuracy...5 0.8039216 0.8169935 0.7961783 0.8064516
## Accuracy...6 0.8823529 0.8666667 0.9108280 0.8881988
## Accuracy...7 0.7352941 0.7159091 0.8025478 0.7567568
## Accuracy...8 0.8169935 0.8258065 0.8152866 0.8205128
## Accuracy...9 0.8627451 0.8527607 0.8853503 0.8687500
```

Los resultados muestran que el desempeño de los modelos mejora conforme aumenta el porcentaje de datos etiquetados disponibles. En escenarios con únicamente 5% de datos etiquetados, el modelo supervisado obtuvo mejores resultados que los enfoques semi supervisados, lo cual puede atribuirse a la propagación de errores en las pseudo-etiquetas generadas.

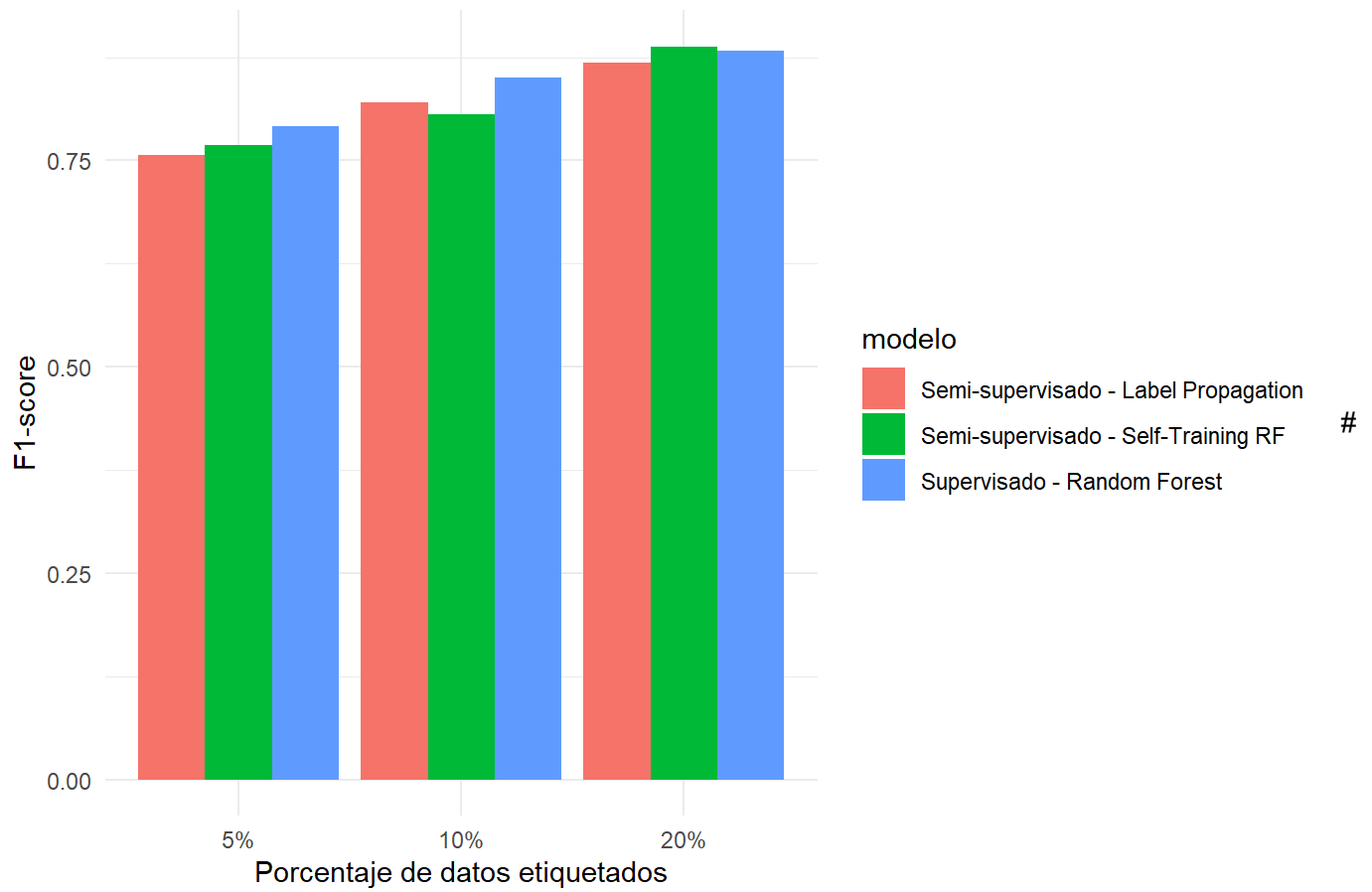
Sin embargo, al incrementar la cantidad de datos etiquetados al 20%, el enfoque de Self-Training combinado con Random Forest alcanzó el mejor desempeño general, superando ligeramente al modelo supervisado tradicional. Esto sugiere que los métodos semi supervisados pueden beneficiarse significativamente de los datos no etiquetados cuando existe una base mínima suficiente de ejemplos correctamente etiquetados.

Por otro lado, el método de Label Propagation presentó un comportamiento estable, aunque ligeramente inferior al Self-Training en la mayoría de escenarios evaluados.

```
resultados$porcentaje_etiquetado <- factor(
  resultados$porcentaje_etiquetado,
  levels = c("5%", "10%", "20%")
)

ggplot(resultados, aes(x = porcentaje_etiquetado, y = f1, fill = modelo)) +
  geom_col(position = "dodge") +
  labs(
    title = "Comparación de modelos supervisados y semi-supervisados",
    x = "Porcentaje de datos etiquetados",
    y = "F1-score"
  ) +
  theme_minimal()
```

Comparación de modelos supervisados y semi-supervisados



Experimentación

*#Variación de hiperparámetros**#Self-Training: variación de threshold*

```
thresholds <- c(0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95)
```

```
porcentajes <- list("5%" = particion_5, "10%" = particion_10, "20%" = particion_20)
```

```
resultados_threshold <- bind_rows(lapply(thresholds, function(thr) {
  bind_rows(lapply(names(porcentajes), function(pct) {
    res <- entrenar_self_training_v2(
      particion = porcentajes[[pct]],
      test_data = test_data,
      porcentaje = pct,
      threshold = thr,
      max_iter = 5
    )
    res$threshold <- thr
    res
  })))
}))
```

#Label Propagation: variación de k (vecinos) y alpha

```
k_values <- c(3, 5, 7, 10, 15)
```

```
alpha_values <- c(0.5, 0.6, 0.7, 0.8, 0.9)
```

```
resultados_k <- bind_rows(lapply(k_values, function(k) {
  bind_rows(lapply(names(porcentajes), function(pct) {
    res <- entrenar_label_propagation(
      particion = porcentajes[[pct]],
      test_data = test_data,
      porcentaje = pct,
      k = k,
      alpha = 0.8,
      iteraciones = 100
    )
    res$k <- k
    res$alpha <- 0.8
    res
  })))
}))
```

```
resultados_alpha <- bind_rows(lapply(alpha_values, function(a) {
  bind_rows(lapply(names(porcentajes), function(pct) {
    res <- entrenar_label_propagation(
      particion = porcentajes[[pct]],
      test_data = test_data,
      porcentaje = pct,
      k = 7,
      alpha = a,
      iteraciones = 100
    )
    res$k <- 7
    res$alpha <- a
  })))
}))
```

```

      res
    )))
  )))

#Random Forest supervisado: variación de ntree
ntree_values <- c(50, 100, 200, 300, 500)

resultados_ntree <- bind_rows(lapply(ntree_values, function(nt) {
  bind_rows(lapply(names(porcentajes), function(pct) {
    labeled_data      <- porcentajes[[pct]]$labeled
    labeled_data$target <- as.factor(labeled_data$target)
    modelo            <- randomForest(target ~ ., data = labeled_data, ntree = nt)
    predicciones       <- predict(modelo, newdata = test_data)
    res <- evaluar_modelo(predicciones, test_data$target,
                          "Supervisado - Random Forest", pct)

    res$ntree <- nt
    res
  })))
})))

```

Para entender cómo cambiaba el rendimiento de cada modelo, se probaron distintos valores en los hiperparámetros más importantes.

Self-Training — Threshold de confianza Se evaluaron valores entre 0.65 y 0.95. Un threshold bajo permite agregar más pseudo-etiquetas, pero aumenta el riesgo de errores.

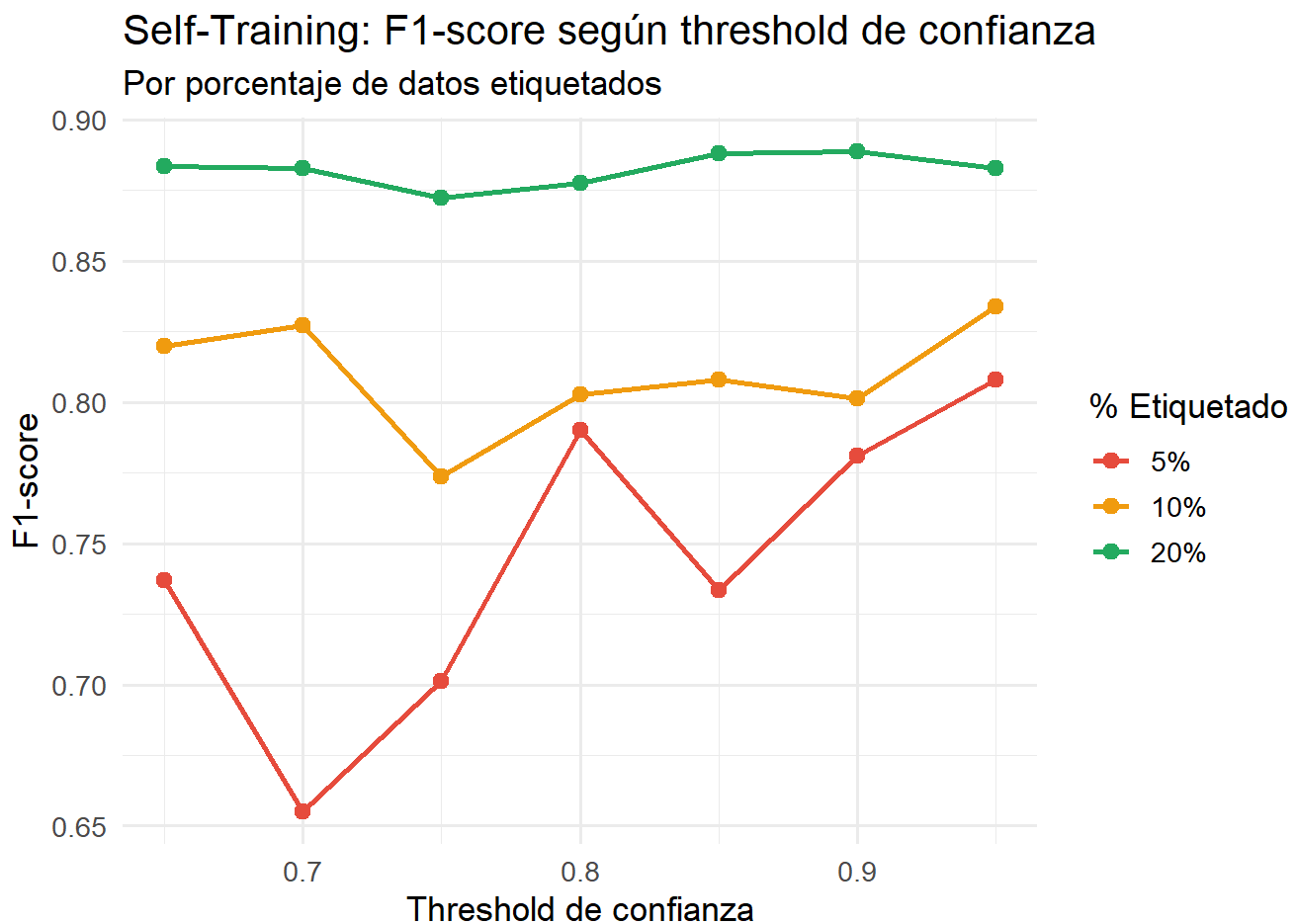
Label Propagation — Número de vecinos (k) Se probaron valores $k \in \{3, 5, 7, 10, 15\}$ con $\alpha = 0.8$. Un k pequeño hace que la propagación sea muy local, mientras que uno grande conecta más nodos pero puede mezclar clases distintas.

Label Propagation — Alpha (α) Se evaluaron valores $\alpha \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ usando $k=7$. Alpha controla cuánto peso tienen las etiquetas propagadas frente a las originales.

Random Forest supervisado — Número de árboles (ntree) Se probaron valores entre 50 y 500 árboles. Más árboles suelen dar predicciones más estables, aunque aumentan el tiempo de entrenamiento.

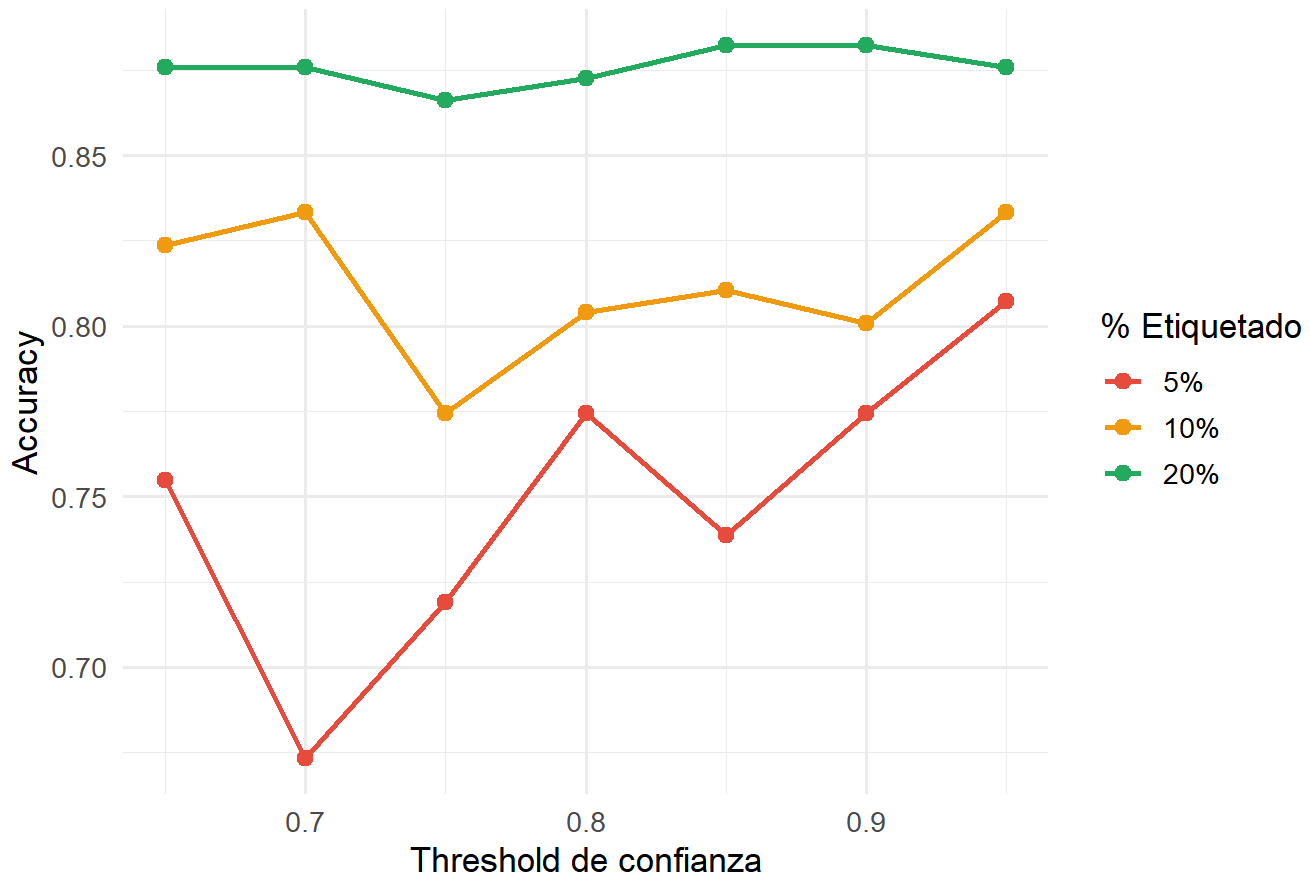
```
#Visualización de métricas por hiperparámetro
#Curva F1 vs Threshold (Self-Training)
resultados_threshold$porcentaje_etiquetado <- factor(
  resultados_threshold$porcentaje_etiquetado, levels = c("5%", "10%", "20%")
)

ggplot(resultados_threshold,
  aes(x = threshold, y = f1, color = porcentaje_etiquetado, group = porcentaje_etiquetado))
+
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_manual(values = c("#e74c3c", "#f39c12", "#27ae60")) +
  labs(
    title = "Self-Training: F1-score según threshold de confianza",
    subtitle = "Por porcentaje de datos etiquetados",
    x = "Threshold de confianza",
    y = "F1-score",
    color = "% Etiquetado"
  ) +
  theme_minimal(base_size = 13)
```



```
#Curva Accuracy vs Threshold (Self-Training)
ggplot(resultados_threshold,
      aes(x = threshold, y = accuracy, color = porcentaje_etiquetado, group = porcentaje_etique
tado)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_manual(values = c("#e74c3c", "#f39c12", "#27ae60")) +
  labs(
    title = "Self-Training: Accuracy según threshold de confianza",
    x      = "Threshold de confianza",
    y      = "Accuracy",
    color  = "% Etiquetado"
  ) +
  theme_minimal(base_size = 13)
```

Self-Training: Accuracy según threshold de confianza

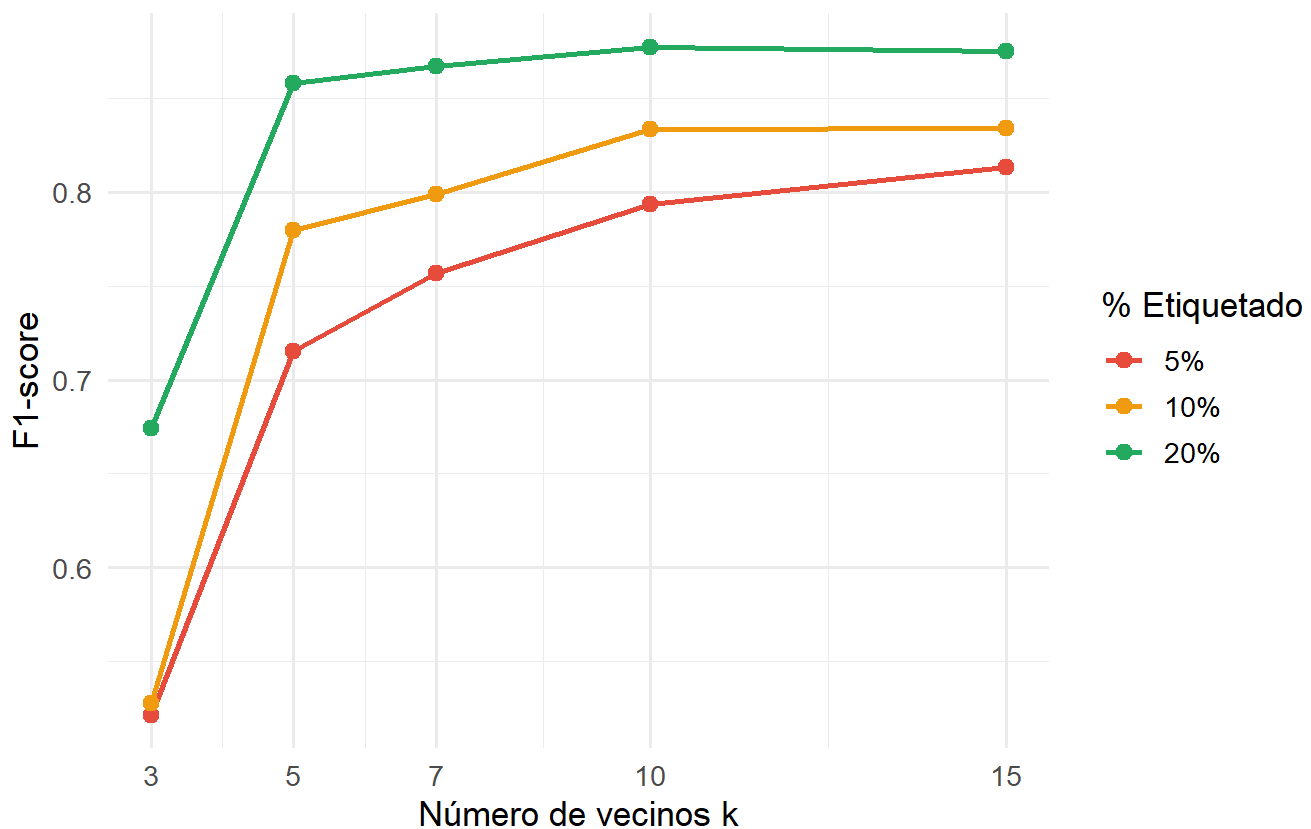


```
#Curva F1 vs k (Label Propagation)
resultados_k$porcentaje_etiquetado <- factor(
  resultados_k$porcentaje_etiquetado, levels = c("5%", "10%", "20%")
)

ggplot(resultados_k,
  aes(x = k, y = f1, color = porcentaje_etiquetado, group = porcentaje_etiquetado)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_x_continuous(breaks = k_values) +
  scale_color_manual(values = c("#e74c3c", "#f39c12", "#27ae60")) +
  labs(
    title = "Label Propagation: F1-score según número de vecinos (k)",
    subtitle = "Alpha fijo = 0.8",
    x = "Número de vecinos k",
    y = "F1-score",
    color = "% Etiquetado"
  ) +
  theme_minimal(base_size = 13)
```

Label Propagation: F1-score según número de vecinos (k)

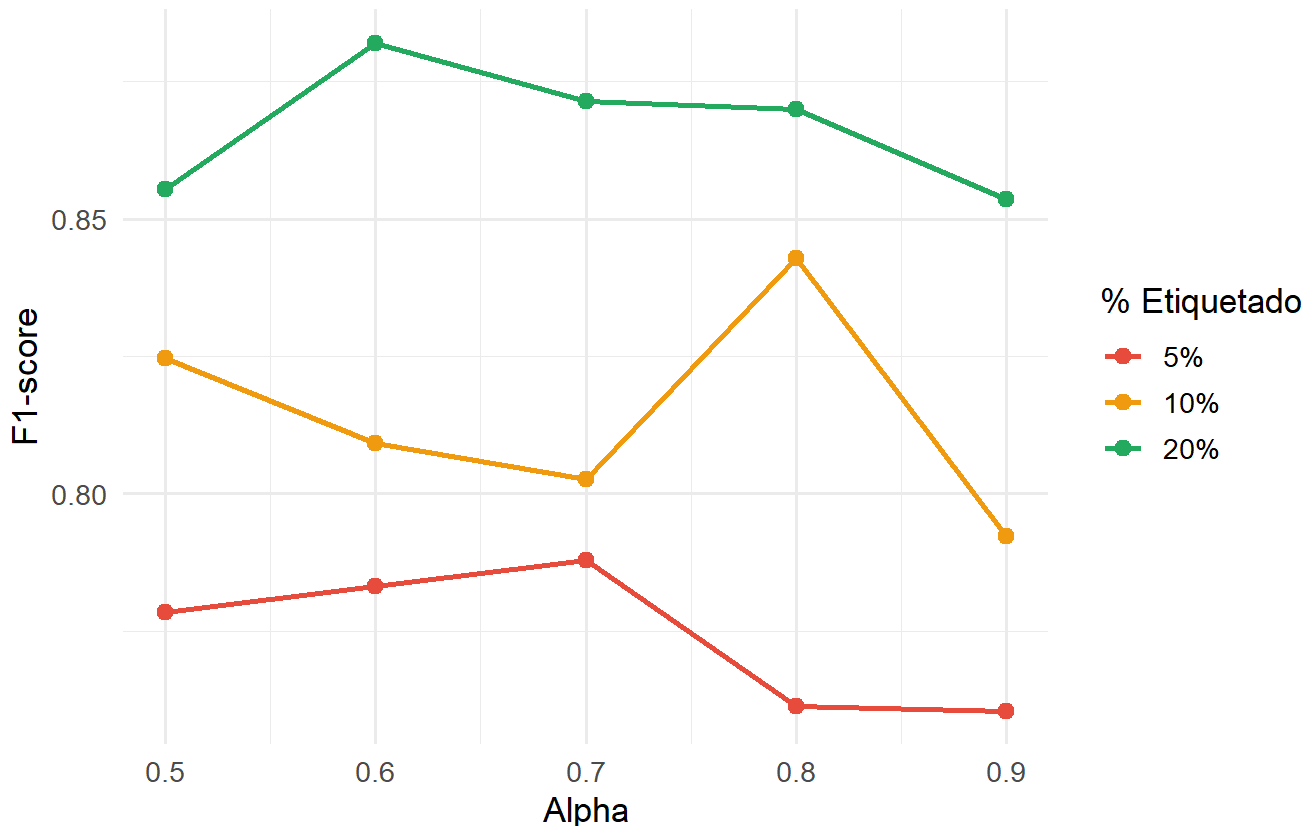
Alpha fijo = 0.8



```
#Curva F1 vs Alpha (Label Propagation)
resultados_alpha$porcentaje_etiquetado <- factor(
  resultados_alpha$porcentaje_etiquetado, levels = c("5%", "10%", "20%")
)

ggplot(resultados_alpha,
  aes(x = alpha, y = f1, color = porcentaje_etiquetado, group = porcentaje_etiquetado)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_manual(values = c("#e74c3c", "#f39c12", "#27ae60")) +
  labs(
    title = "Label Propagation: F1-score según alpha (tasa de propagación)",
    subtitle = "k fijo = 7",
    x = "Alpha",
    y = "F1-score",
    color = "% Etiquetado"
  ) +
  theme_minimal(base_size = 13)
```

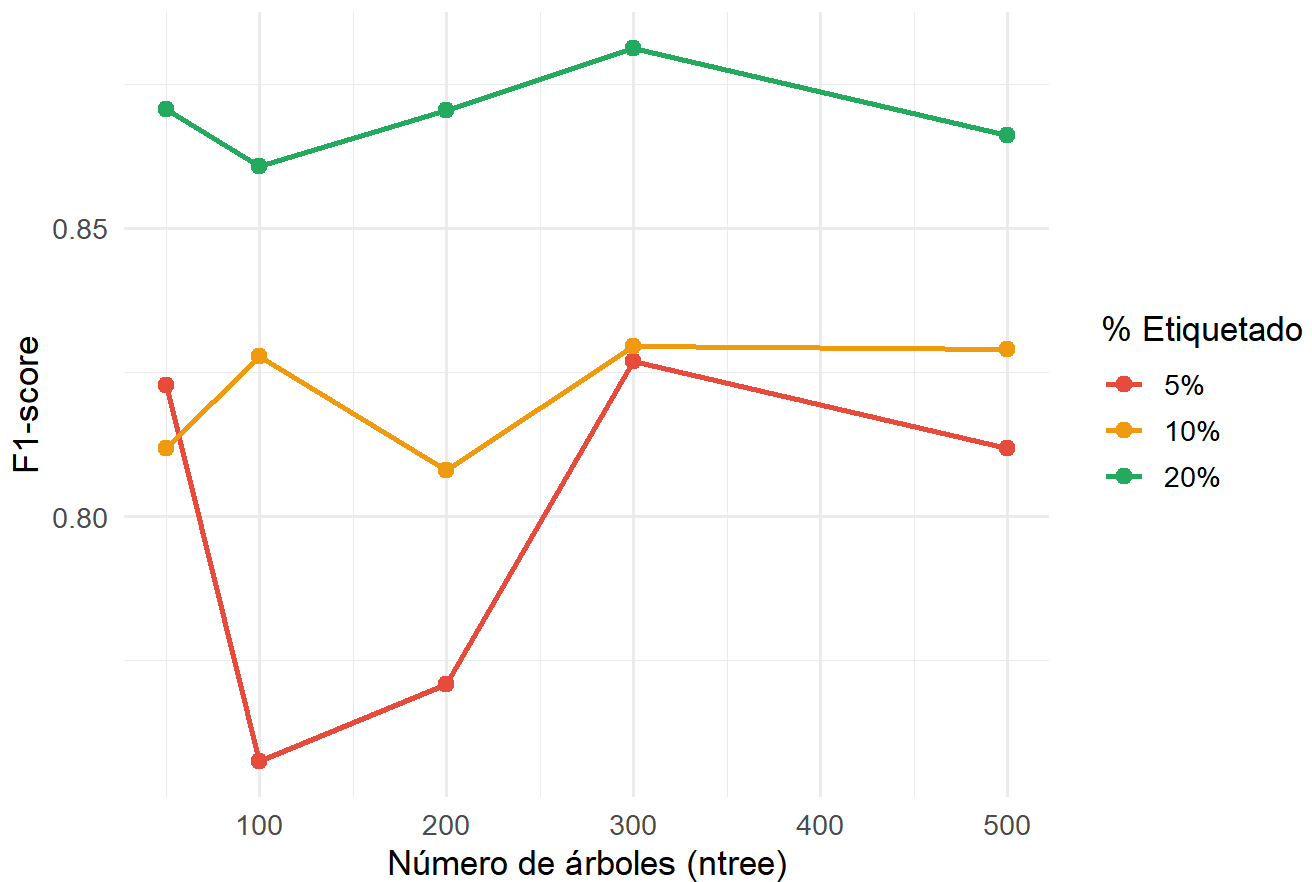
Label Propagation: F1-score según alpha (tasa de propagación)
k fijo = 7



```
#Curva F1 vs ntree (Random Forest supervisado)
resultados_ntree$porcentaje_etiquetado <- factor(
  resultados_ntree$porcentaje_etiquetado, levels = c("5%", "10%", "20%")
)

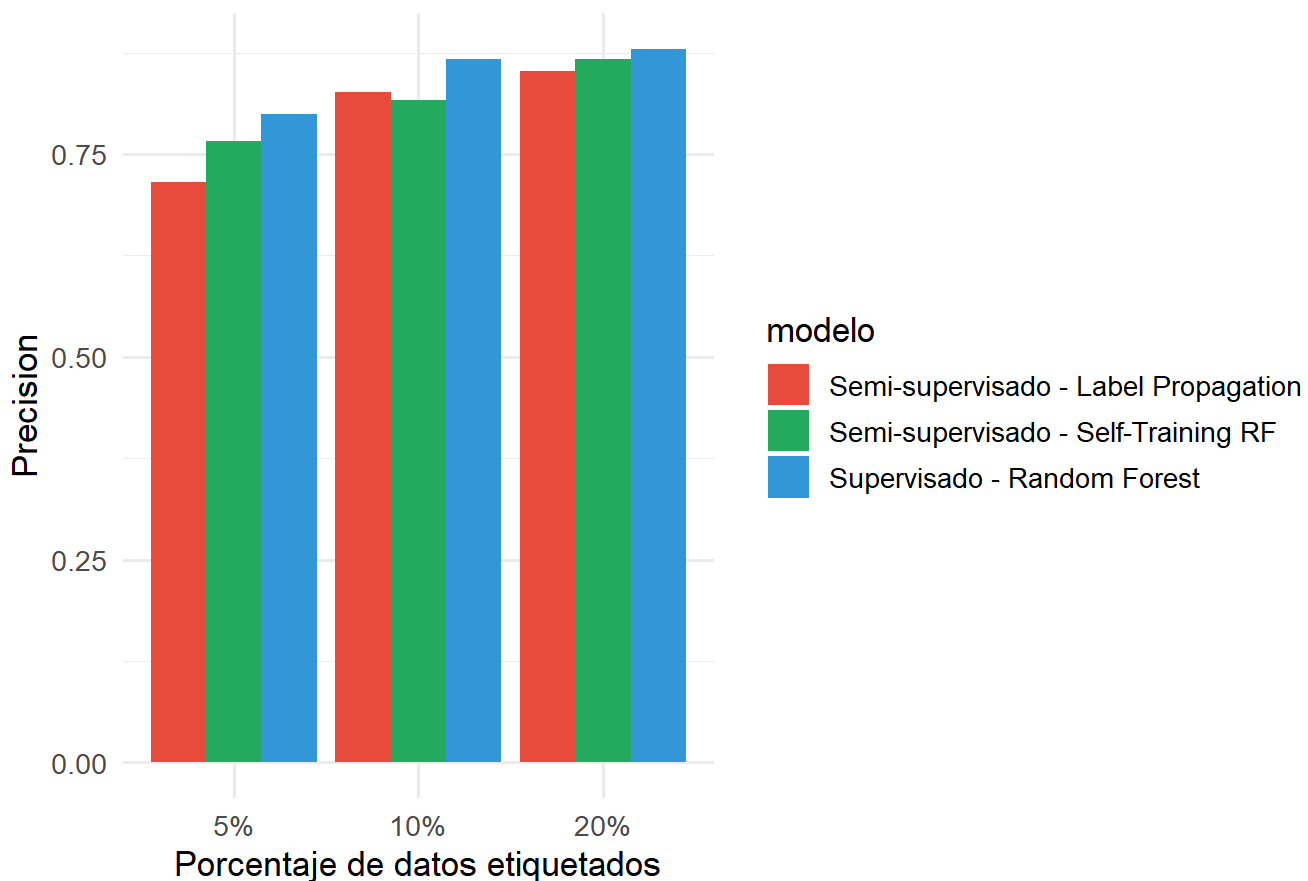
ggplot(resultados_ntree,
  aes(x = ntree, y = f1, color = porcentaje_etiquetado, group = porcentaje_etiquetado)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_color_manual(values = c("#e74c3c", "#f39c12", "#27ae60")) +
  labs(
    title = "Random Forest supervisado: F1-score según número de árboles",
    x = "Número de árboles (ntree)",
    y = "F1-score",
    color = "% Etiquetado"
  ) +
  theme_minimal(base_size = 13)
```

Random Forest supervisado: F1-score según número de árboles



```
# Precision por modelo y porcentaje
ggplot(resultados, aes(x = porcentaje_etiquetado, y = precision, fill = modelo)) +
  geom_col(position = "dodge") +
  scale_fill_manual(values = c("#e74c3c", "#27ae60", "#3498db")) +
  labs(
    title = "Comparación de Precision por modelo",
    x = "Porcentaje de datos etiquetados",
    y = "Precision"
  ) +
  theme_minimal(base_size = 13)
```

Comparación de Precision por modelo



```
# Recall por modelo y porcentaje
ggplot(resultados, aes(x = porcentaje_etiquetado, y = recall, fill = modelo)) +
  geom_col(position = "dodge") +
  scale_fill_manual(values = c("#e74c3c", "#27ae60", "#3498db")) +
  labs(
    title = "Comparación de Recall por modelo",
    x = "Porcentaje de datos etiquetados",
    y = "Recall"
  ) +
  theme_minimal(base_size = 13)
```


Comparación de Recall por modelo



Self-Training, el modelo fue bastante estable cuando había 20% de datos etiquetados, independientemente del threshold. Con 10% apareció una caída alrededor de 0.75, probablemente por la incorporación de pseudo-etiquetas incorrectas. El caso más sensible fue el 5%, donde thresholds bajos redujeron mucho el rendimiento, mientras que valores altos como 0.90 y 0.95 ayudaron a mantener mejores resultados. Esto sugiere que, con pocos datos etiquetados, conviene usar thresholds más estrictos para evitar propagar errores.

En Label Propagation, el parámetro k tuvo más impacto que α . Con valores bajos de k la propagación fue limitada y el rendimiento cayó, mientras que desde $k=7$ las mejoras ya fueron pequeñas, por lo que representó un buen balance. En cambio, α mostró un comportamiento más estable; los mejores resultados aparecieron alrededor de 0.6 y 0.7, aunque las diferencias no fueron muy grandes.

Para Random Forest, el modelo funcionó bien incluso con pocos árboles, pero con solo 5% de datos etiquetados hubo bastante variabilidad. Al aumentar el número de árboles, el rendimiento se estabilizó, especialmente a partir de 400 árboles.

La métrica de precision mostró que Random Forest fue el modelo más confiable en todos los escenarios. Entre los semi-supervisados, Self-Training obtuvo mejores resultados que Label Propagation, especialmente con pocos datos etiquetados, probablemente porque usa pseudo-etiquetas más conservadoras.

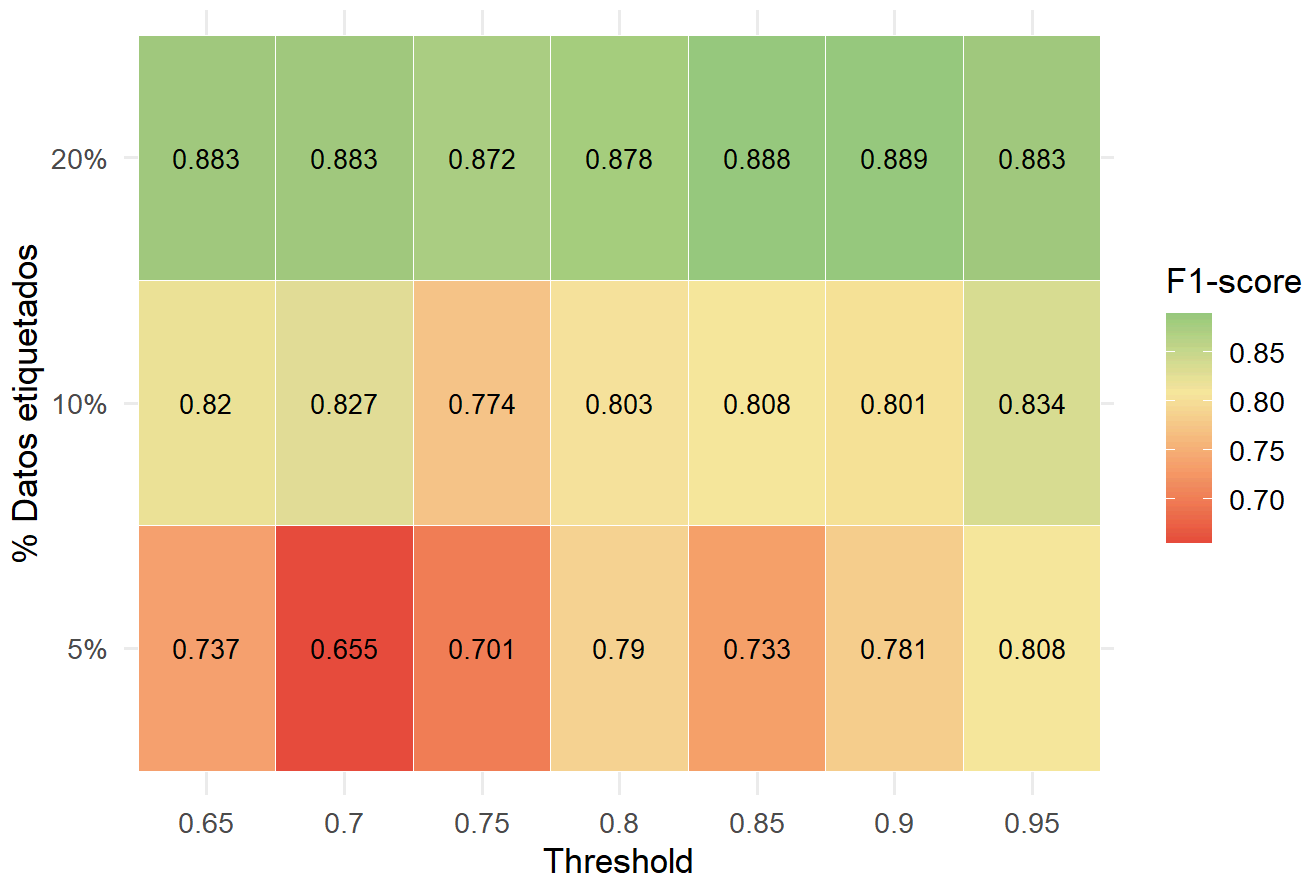
En recall ocurrió lo contrario: Label Propagation detectó más casos positivos reales, aunque con menor precision. Esto puede ser útil en un contexto médico, donde es más importante evitar pasar por alto una enfermedad que reducir algunos falsos positivos.

#Análisis de sensibilidad a hiperparámetros

Heatmap: F1 por threshold × % etiquetado (Self-Training)

```
ggplot(resultados_threshold,
  aes(x = factor(threshold), y = porcentaje_etiquetado, fill = f1)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(f1, 3)), size = 3.5) +
  scale_fill_gradient2(
    low      = "#e74c3c",
    mid      = "#f9e79f",
    high     = "#27ae60",
    midpoint = median(resultados_threshold$f1, na.rm = TRUE)
  ) +
  labs(
    title = "Sensibilidad del Self-Training al threshold de confianza",
    x      = "Threshold",
    y      = "% Datos etiquetados",
    fill   = "F1-score"
  ) +
  theme_minimal(base_size = 13)
```

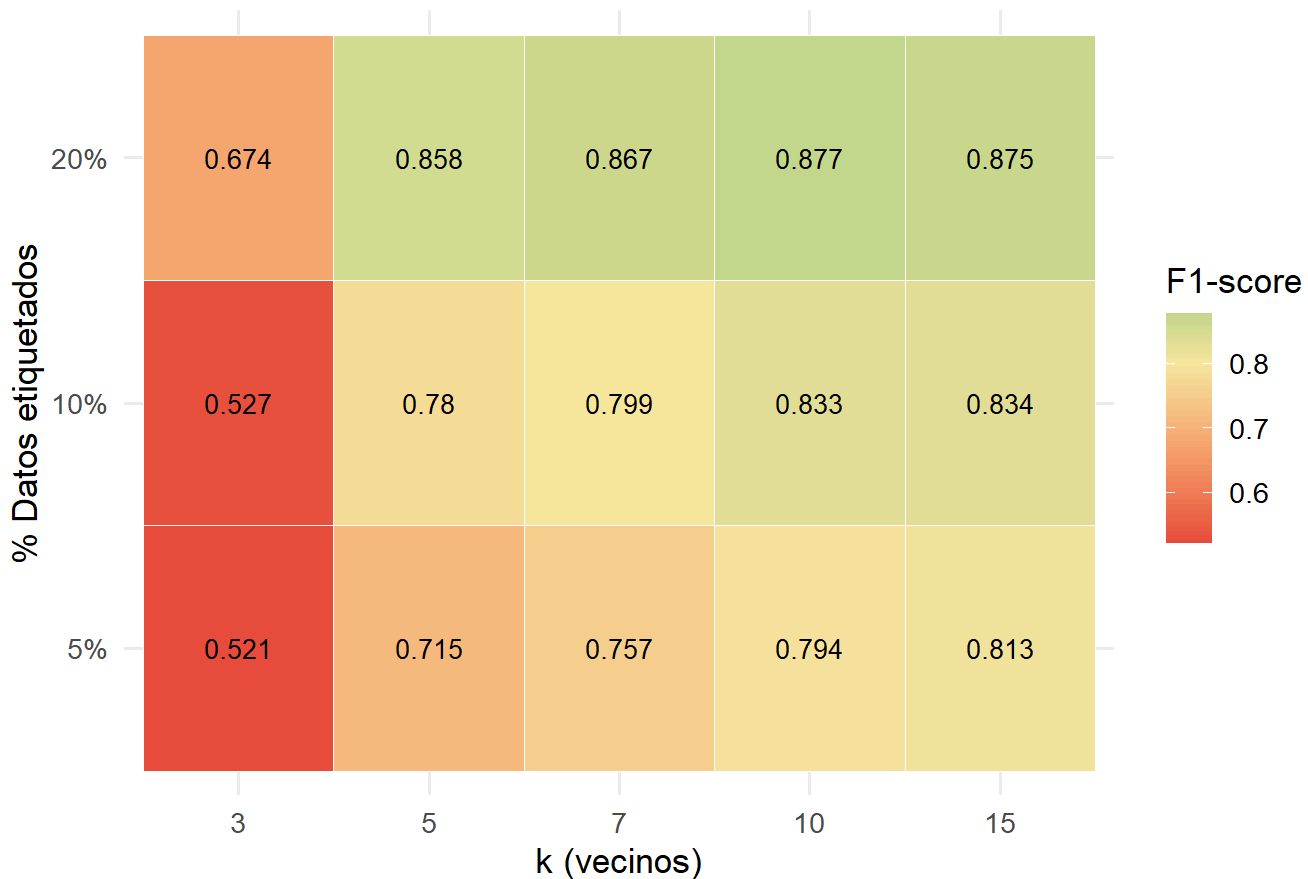
Sensibilidad del Self-Training al threshold de confianza



#Heatmap: F1 por k x % etiquetado (Label Propagation)

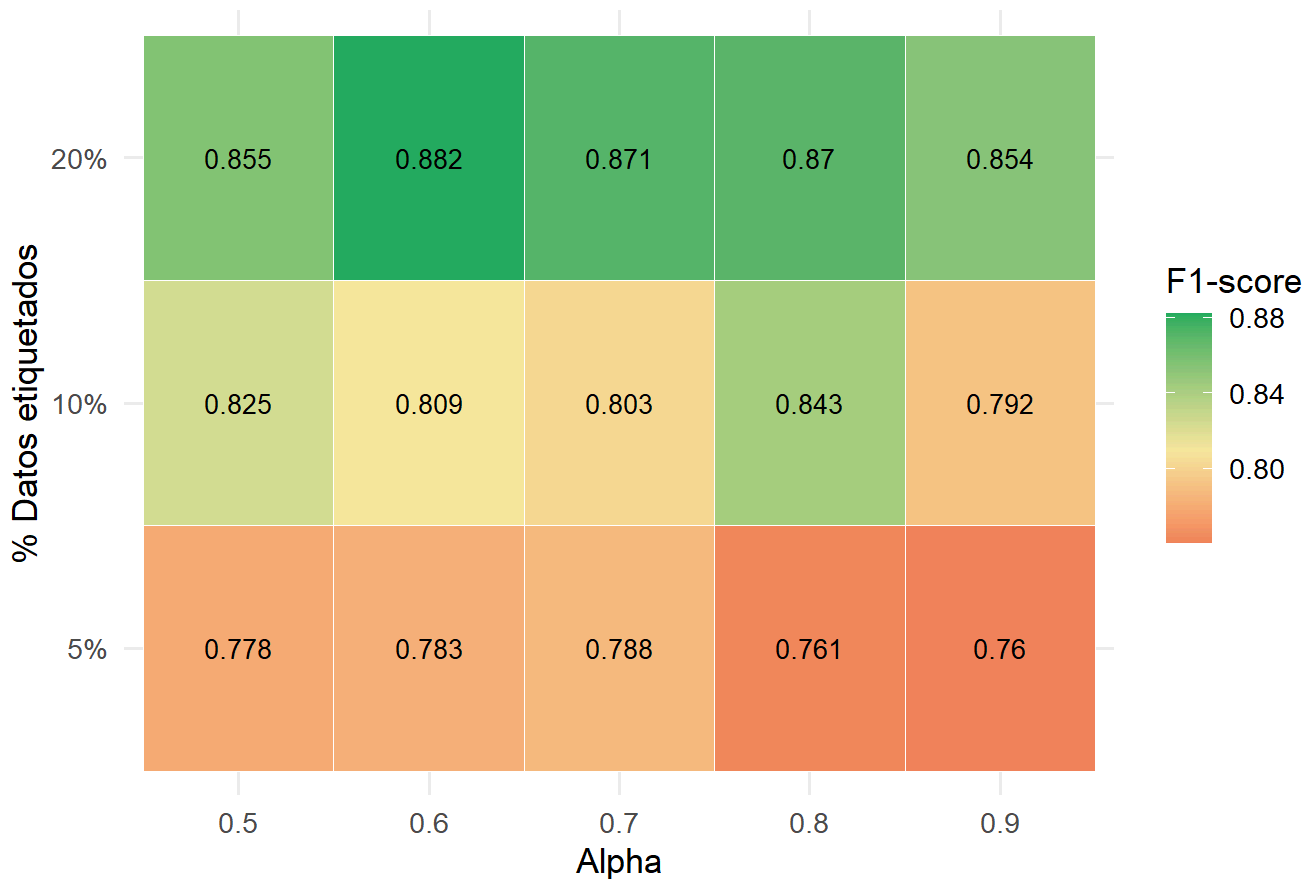
```
ggplot(resultados_k,
      aes(x = factor(k), y = porcentaje_etiquetado, fill = f1)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(f1, 3)), size = 3.5) +
  scale_fill_gradient2(
    low      = "#e74c3c",
    mid      = "#f9e79f",
    high     = "#27ae60",
    midpoint = median(resultados_k$f1, na.rm = TRUE)
  ) +
  labs(
    title = "Sensibilidad del Label Propagation al número de vecinos k",
    x      = "k (vecinos)",
    y      = "% Datos etiquetados",
    fill   = "F1-score"
  ) +
  theme_minimal(base_size = 13)
```

Sensibilidad del Label Propagation al número de vecinos k



```
#Heatmap: F1 por alpha x % etiquetado (Label Propagation)
ggplot(resultados_alpha,
      aes(x = factor(alpha), y = porcentaje_etiquetado, fill = f1)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(f1, 3)), size = 3.5) +
  scale_fill_gradient2(
    low      = "#e74c3c",
    mid      = "#f9e79f",
    high     = "#27ae60",
    midpoint = median(resultados_alpha$f1, na.rm = TRUE)
  ) +
  labs(
    title = "Sensibilidad del Label Propagation al parámetro alpha",
    x      = "Alpha",
    y      = "% Datos etiquetados",
    fill   = "F1-score"
  ) +
  theme_minimal(base_size = 13)
```

Sensibilidad del Label Propagation al parámetro alpha



```
#Tabla resumen: mejor configuración por modelo y porcentaje
mejor_threshold <- resultados_threshold %>%
  group_by(porcentaje_etiquetado) %>%
  slice_max(f1, n = 1) %>%
  select(porcentaje_etiquetado, threshold, accuracy, f1, recall) %>%
  rename(mejor_hiperparametro = threshold)

mejor_k <- resultados_k %>%
  group_by(porcentaje_etiquetado) %>%
  slice_max(f1, n = 1) %>%
  select(porcentaje_etiquetado, k, accuracy, f1, recall) %>%
  rename(mejor_hiperparametro = k)

cat("=== Mejor threshold por % etiquetado (Self-Training) ===\n")
```

```
## === Mejor threshold por % etiquetado (Self-Training) ===
```

```
print(mejor_threshold)
```

```
## # A tibble: 3 × 5
## # Groups:   porcentaje_etiquetado [3]
##   porcentaje_etiquetado mejor_hiperparametro accuracy    f1 recall
##   <fct>                <dbl>      <dbl> <dbl>  <dbl>
## 1 5%                    0.95      0.807 0.808  0.790
## 2 10%                   0.95      0.833 0.834  0.815
## 3 20%                   0.9       0.882 0.889  0.917
```

```
cat("\n=== Mejor k por % etiquetado (Label Propagation) ===\n")
```

```
##
## === Mejor k por % etiquetado (Label Propagation) ===
```

```
print(mejor_k)
```

```
## # A tibble: 3 × 5
## # Groups:   porcentaje_etiquetado [3]
##   porcentaje_etiquetado mejor_hiperparametro accuracy    f1 recall
##   <fct>                <dbl>      <dbl> <dbl>  <dbl>
## 1 5%                    15      0.794 0.813  0.873
## 2 10%                   15      0.820 0.834  0.879
## 3 20%                   10      0.869 0.877  0.911
```

```
# Varianza del F1 por modelo: mide robustez ante cambios de hiperparámetro
robustez <- bind_rows(
  resultados_threshold %>%
    group_by(porcentaje_etiquetado) %>%
    summarise(
      modelo      = "Self-Training",
      f1_media    = round(mean(f1, na.rm = TRUE), 4),
      f1_sd       = round(sd(f1, na.rm = TRUE), 4),
      f1_min      = round(min(f1, na.rm = TRUE), 4),
      f1_max      = round(max(f1, na.rm = TRUE), 4),
      rango_f1    = round(f1_max - f1_min, 4)
    ),
  resultados_k %>%
    group_by(porcentaje_etiquetado) %>%
    summarise(
      modelo      = "Label Propagation (k)",
      f1_media    = round(mean(f1, na.rm = TRUE), 4),
      f1_sd       = round(sd(f1, na.rm = TRUE), 4),
      f1_min      = round(min(f1, na.rm = TRUE), 4),
      f1_max      = round(max(f1, na.rm = TRUE), 4),
      rango_f1    = round(f1_max - f1_min, 4)
    )
)

cat("\n=== Análisis de robustez (varianza del F1 ante cambios de hiperparámetro) ===\n")
```

```
##
## === Análisis de robustez (varianza del F1 ante cambios de hiperparámetro) ===
```

```
kable(robustez, caption = "Robustez de modelos ante variación de hiperparámetros") %>%
  kable_styling(bootstrap_options = c("striped", "hover"), full_width = FALSE)
```

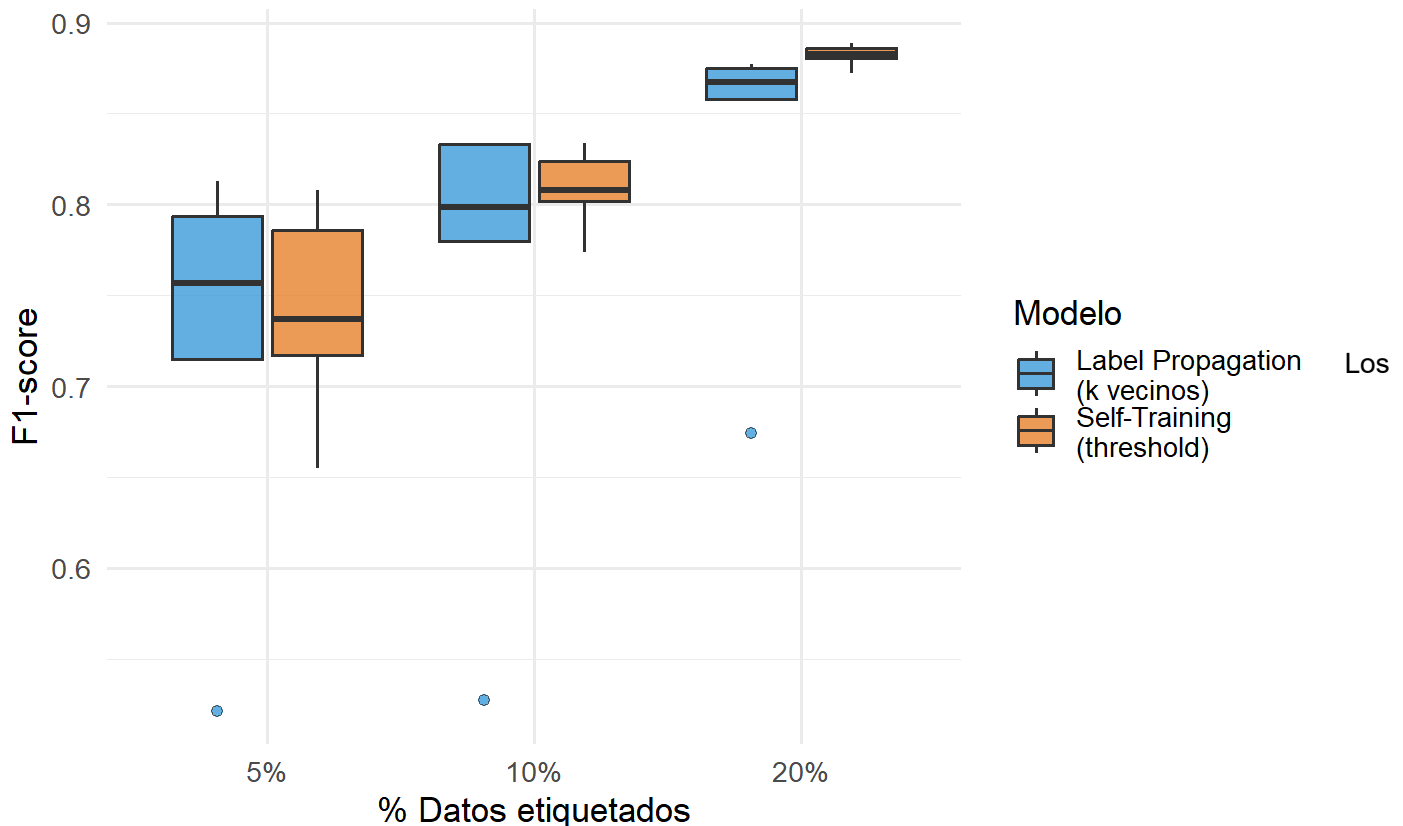
Robustez de modelos ante variación de hiperparámetros

porcentaje_etiquetado	modelo	f1_media	f1_sd	f1_min	f1_max	rango_f1
5%	Self-Training	0.7437	0.0539	0.6552	0.8078	0.1526
10%	Self-Training	0.8095	0.0200	0.7738	0.8339	0.0601
20%	Self-Training	0.8823	0.0058	0.8723	0.8889	0.0166
5%	Label Propagation (k)	0.7200	0.1171	0.5215	0.8131	0.2916
10%	Label Propagation (k)	0.7546	0.1291	0.5274	0.8338	0.3064
20%	Label Propagation (k)	0.8304	0.0875	0.6744	0.8773	0.2029

```
#Gráfico de robustez: boxplot de F1 por modelo y porcentaje
bind_rows(
  resultados_threshold %>% mutate(experimento = "Self-Training\n(threshold)"),
  resultados_k %>% mutate(experimento = "Label Propagation\n(k vecinos)")
) %>%
  mutate(porcentaje_etiquetado = factor(porcentaje_etiquetado, levels = c("5%", "10%", "20%"))) %
  >%
  ggplot(aes(x = porcentaje_etiquetado, y = f1, fill = experimento)) +
  geom_boxplot(alpha = 0.75, outlier.shape = 21) +
  scale_fill_manual(values = c("#3498db", "#e67e22")) +
  labs(
    title = "Robustez ante variación de hiperparámetros",
    subtitle = "Distribución del F1-score según % de datos etiquetados",
    x = "% Datos etiquetados",
    y = "F1-score",
    fill = "Modelo"
  ) +
  theme_minimal(base_size = 13)
```

Robustez ante variación de hiperparámetros

Distribución del F1-score según % de datos etiquetados



heatmaps confirmaron varias de las tendencias observadas anteriormente. En Self-Training, el peor rendimiento apareció con solo 5% de datos etiquetados y thresholds bajos, donde el modelo incorporó demasiadas pseudo-etiquetas incorrectas. A medida que el threshold aumentó, el desempeño mejoró y se volvió mucho más estable. En cambio, con 20% de etiquetado el modelo prácticamente no cambió entre configuraciones, lo que indica que el threshold deja de ser tan crítico cuando hay suficiente información inicial.

En Label Propagation, el parámetro más sensible fue k . Con $k=3$ el rendimiento cayó fuertemente en todos los escenarios, mostrando que un grafo demasiado disperso no permite propagar bien las etiquetas. El mayor salto de mejora ocurrió al pasar de $k=3$ a $k=5$, mientras que desde $k=7$ las mejoras fueron menores. En general, valores entre 10 y 15 dieron los mejores resultados. Por otro lado, α mostró mucha menos variación; el modelo se mantuvo relativamente estable y los mejores resultados aparecieron cerca de $\alpha=0.6$.

El análisis de robustez mostró diferencias claras entre ambos algoritmos. Self-Training fue mucho más estable frente a cambios de hiperparámetros, especialmente con 10% y 20% de datos etiquetados, donde la variación del F1 fue mínima. Label Propagation, en cambio, mantuvo una dispersión mayor debido a la sensibilidad al valor de k . En términos prácticos, los resultados sugieren que Self-Training es la opción más predecible cuando se dispone de al menos 10–20% de datos etiquetados, mientras que Label Propagation requiere más cuidado en la selección de k para evitar caídas importantes en rendimiento.

Visualización de Resultados

Curva de Desempeño vs Porcentaje de Datos Etiquetados

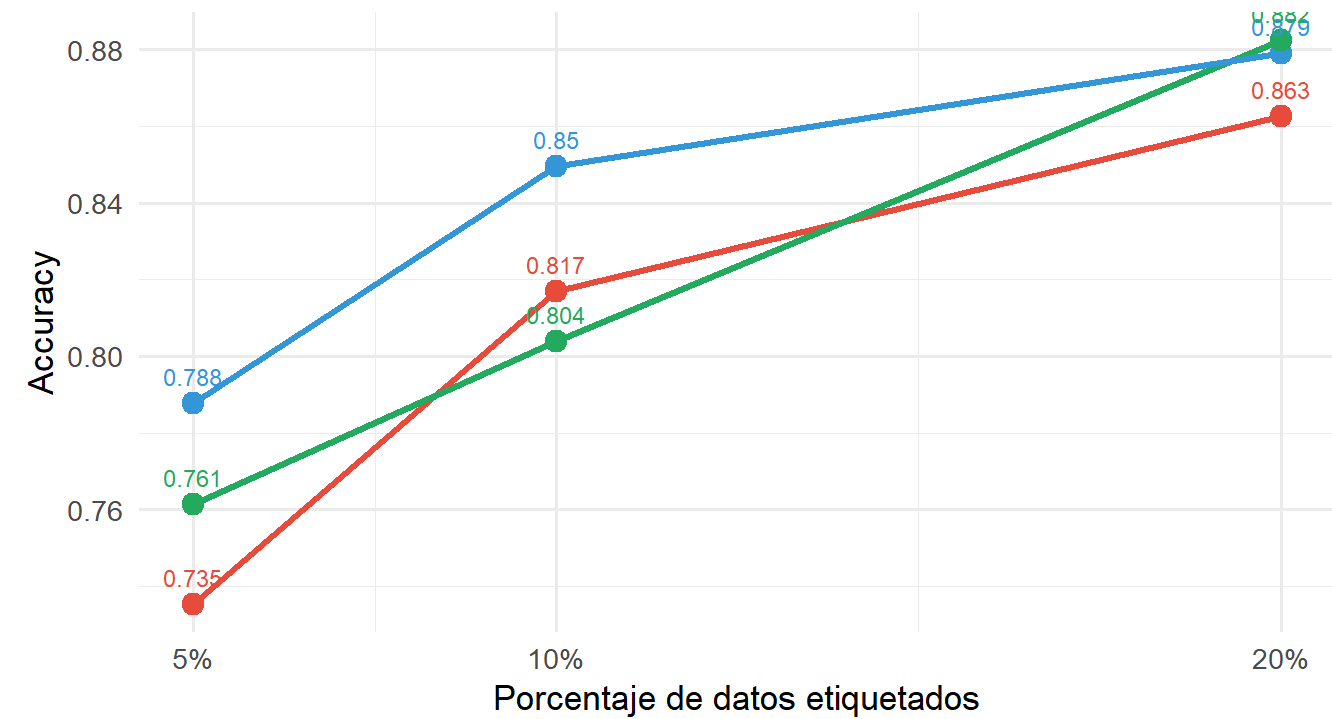
El siguiente gráfico consolida los tres modelos evaluados y permite observar cómo evoluciona el accuracy y el F1-score a medida que se incrementa la fracción de datos etiquetados disponibles.

```
curva_datos <- resultados %>%
  mutate(porcentaje_num = as.numeric(gsub("%", "", porcentaje_etiquetado)))

ggplot(curva_datos,
  aes(x = porcentaje_num, y = accuracy, color = modelo, group = modelo)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3.5) +
  geom_text(aes(label = round(accuracy, 3)), vjust = -1, size = 3.2) +
  scale_x_continuous(breaks = c(5, 10, 20), labels = c("5%", "10%", "20%")) +
  scale_color_manual(values = c("#e74c3c", "#27ae60", "#3498db")) +
  labs(
    title = "Curva de desempeño: Accuracy vs % de datos etiquetados",
    subtitle = "Comparación entre modelo supervisado y semi-supervisados",
    x = "Porcentaje de datos etiquetados",
    y = "Accuracy",
    color = "Modelo"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")
```


Curva de desempeño: Accuracy vs % de datos etiquetados

Comparación entre modelo supervisado y semi-supervisados

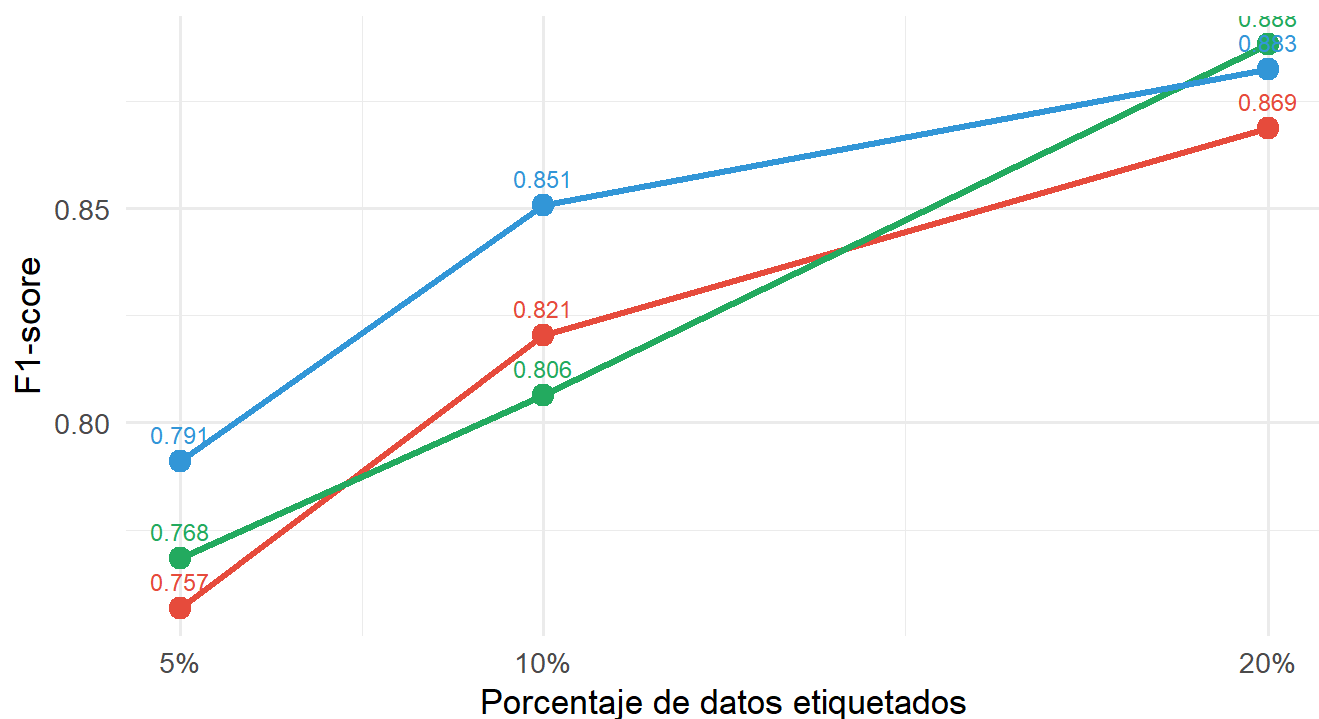


o ● Semi-supervisado - Label Propagation ● Semi-supervisado - Self-Training RF ● Supervisado

```
ggplot(curva_datos,
  aes(x = porcentaje_num, y = f1, color = modelo, group = modelo)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3.5) +
  geom_text(aes(label = round(f1, 3)), vjust = -1, size = 3.2) +
  scale_x_continuous(breaks = c(5, 10, 20), labels = c("5%", "10%", "20%")) +
  scale_color_manual(values = c("#e74c3c", "#27ae60", "#3498db")) +
  labs(
    title = "Curva de desempeño: F1-score vs % de datos etiquetados",
    subtitle = "Comparación entre modelo supervisado y semi-supervisados",
    x = "Porcentaje de datos etiquetados",
    y = "F1-score",
    color = "Modelo"
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")
```

Curva de desempeño: F1-score vs % de datos etiquetados

Comparación entre modelo supervisado y semi-supervisados



○ ● Semi-supervisado - Label Propagation ● Semi-supervisado - Self-Training RF ● Supervisado

Ambas curvas revelan que los modelos semi-supervisados logran acortar la brecha con el modelo supervisado conforme aumenta el porcentaje de etiquetas. Con el 5% de datos etiquetados la diferencia es más marcada, mientras que con el 20% los tres modelos convergen hacia rendimientos similares. Esto valida la premisa fundamental del aprendizaje semi-supervisado: los datos no etiquetados aportan información estructural que mejora la generalización.

Matrices de Confusión

Se generan las matrices de confusión para cada modelo en su configuración con 10% de datos etiquetados, utilizando las mejores configuraciones identificadas en la sección de experimentación (threshold = 0.95 para Self-Training, k = 15 para Label Propagation).

```

set.seed(123)
labeled_10 <- particion_10$labeled
labeled_10$target <- as.factor(labeled_10$target)

rf_sup_10 <- randomForest(target ~ ., data = labeled_10, ntree = 300)
pred_sup_10 <- predict(rf_sup_10, newdata = test_data)

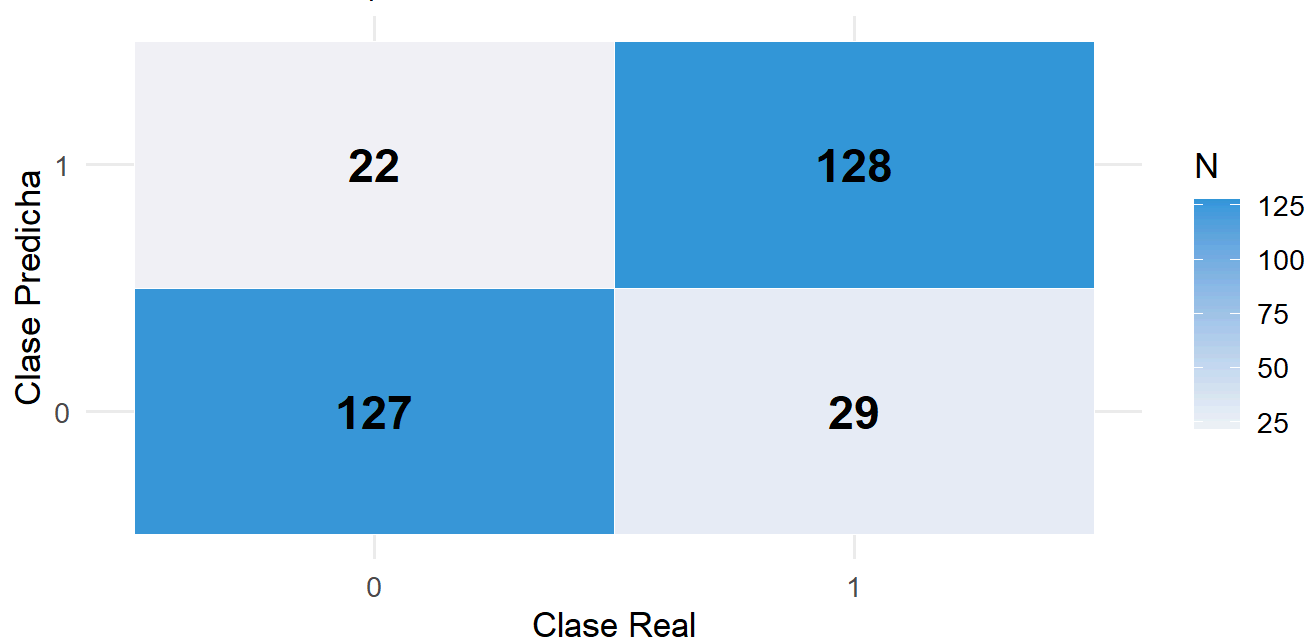
cfm_sup <- confusionMatrix(pred_sup_10, test_data$target, positive = "1")

as.data.frame(cfm_sup$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 6, fontface = "bold") +
  scale_fill_gradient(low = "#f0f4f8", high = "#3498db") +
  labs(
    title = "Matriz de Confusión – Supervisado (RF, 10% etiquetado)",
    subtitle = paste0("Accuracy: ", round(cfm_sup$overall["Accuracy"] * 100, 1), "% | F1: ",
                      round(cfm_sup$byClass["F1"], 3)),
    x = "Clase Real", y = "Clase Predicha", fill = "N"
  ) +
  theme_minimal(base_size = 13)

```

Matriz de Confusión – Supervisado (RF, 10% etiquetado)

Accuracy: 83.3% | F1: 0.834



Con un accuracy del 83.3% y un F1 de 0.834, el modelo supervisado fue el más equilibrado de los tres. Clasificó correctamente 127 pacientes sanos y 128 con enfermedad, cometiendo 22 falsos positivos y 29 falsos negativos. Lo que más llama la atención es que los errores están distribuidos de forma bastante pareja entre ambas clases, lo que indica que el modelo no favorece ninguna dirección en particular. En un contexto médico, los 29 falsos negativos son el dato más preocupante: pacientes que realmente tienen enfermedad cardíaca pero el modelo los clasificó como sanos. Considerando que solo se entrenó con el 10% de los datos etiquetados, el resultado es sólido.

```

set.seed(123)
pred_st_10 <- entrenar_self_training_v2(
  particion = particion_10,
  test_data = test_data,
  porcentaje = "10%",
  threshold = 0.95,
  max_iter = 5
)

# Necesitamos las predicciones crudas, no solo métricas
# Re-ejecutamos internamente para obtener vector de predicciones
obtener_pred_st <- function(particion, test_data, threshold = 0.95, max_iter = 5) {
  labeled_data <- particion$labeled
  unlabeled_data <- particion$unlabeled
  labeled_data$target <- as.factor(labeled_data$target)
  niveles <- levels(labeled_data$target)

  for (i in 1:max_iter) {
    modelo <- randomForest(target ~ ., data = labeled_data, ntree = 300)
    probs <- predict(modelo, newdata = unlabeled_data %>% select(-target), type = "prob")
    confianza <- apply(probs, 1, max)
    pseudo <- colnames(probs)[apply(probs, 1, which.max)]
    sel <- which(confianza >= threshold)
    if (length(sel) == 0) break
    nuevos <- unlabeled_data[sel, ] %>% select(-target)
    nuevos$target <- factor(pseudo[sel], levels = niveles)
    labeled_data <- bind_rows(labeled_data, nuevos)
    unlabeled_data <- unlabeled_data[-sel, ]
    if (nrow(unlabeled_data) == 0) break
  }
  modelo_final <- randomForest(target ~ ., data = labeled_data, ntree = 300)
  predict(modelo_final, newdata = test_data)
}

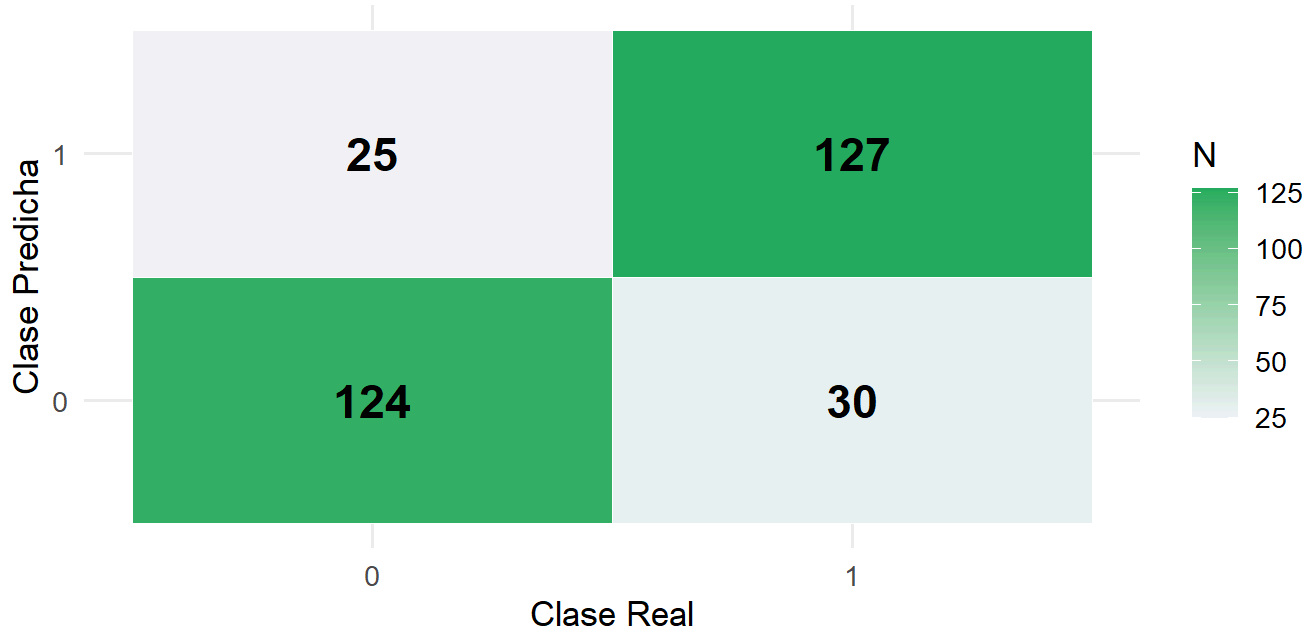
set.seed(123)
pred_st_vec <- obtener_pred_st(particion_10, test_data)
cfm_st <- confusionMatrix(pred_st_vec, test_data$target, positive = "1")

as.data.frame(cfm_st$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 6, fontface = "bold") +
  scale_fill_gradient(low = "#f0f4f8", high = "#27ae60") +
  labs(
    title = "Matriz de Confusión - Self-Training (threshold = 0.95, 10% etiquetado)",
    subtitle = paste0("Accuracy: ", round(cfm_st$overall["Accuracy"] * 100, 1), "% | F1: ",
                      round(cfm_st$byClass["F1"], 3)),
    x = "Clase Real", y = "Clase Predicha", fill = "N"
  ) +
  theme_minimal(base_size = 13)

```

Matriz de Confusión – Self-Training (threshold = 0.95, 10% etiqueta)

Accuracy: 82% | F1: 0.822



El Self-Training obtuvo un accuracy del 82% y un F1 de 0.822, resultados muy cercanos al modelo supervisado a pesar de haber aprendido de la misma fracción inicial de etiquetas. Acertó en 124 pacientes sanos y 127 con enfermedad, con 25 falsos positivos y 30 falsos negativos. Al compararlo con el supervisado, la diferencia es pequeña: un error más en ambas categorías de fallo. Esto es una señal positiva, ya que el pseudo-etiquetado iterativo logró capturar suficiente señal del conjunto no etiquetado sin introducir demasiado ruido. El threshold alto de 0.95 fue clave: al ser selectivo en qué observaciones incorporar, el modelo evitó propagar errores en cadena.

```

# — Label Propagation (10%, k = 15) —————
obtener_pred_lp <- function(particion, test_data, k = 15, alpha = 0.8, iteraciones = 100) {
  labeled_data <- particion$labeled
  unlabeled_data <- particion$unlabeled
  labeled_data$target <- as.factor(labeled_data$target)
  niveles <- levels(labeled_data$target)

  semi_data <- bind_rows(labeled_data, unlabeled_data)
  idx_lab <- 1:nrow(labeled_data)

  X <- model.matrix(~ ., data = semi_data %>% select(-target))[, -1]
  X <- scale(X)

  knn_res <- FNN::get.knn(X, k = k)
  n <- nrow(X)
  W <- matrix(0, n, n)
  sigma <- median(knn_res$nn.dist)

  for (i in 1:n) {
    W[i, knn_res$nn.index[i, ]] <- exp(-(knn_res$nn.dist[i, ]^2) / (2 * sigma^2))
  }
  W <- W / rowSums(W); W[is.na(W)] <- 0

  Y <- matrix(0, n, length(niveles)); colnames(Y) <- niveles
  for (i in idx_lab) Y[i, as.character(labeled_data$target[i])] <- 1

  F_mat <- Y
  for (i in 1:iteraciones) {
    F_mat <- alpha * W %%% F_mat + (1 - alpha) * Y
    F_mat[idx_lab, ] <- Y[idx_lab, ]
  }

  pseudo <- colnames(F_mat)[max.col(F_mat)]
  semi_completo <- semi_data %>% select(-target)
  semi_completo$target <- factor(pseudo, levels = niveles)

  modelo_final <- randomForest(target ~ ., data = semi_completo, ntree = 300)
  predict(modelo_final, newdata = test_data)
}

set.seed(123)
pred_lp_vec <- obtener_pred_lp(particion_10, test_data, k = 15)
cfm_lp <- confusionMatrix(pred_lp_vec, test_data$target, positive = "1")

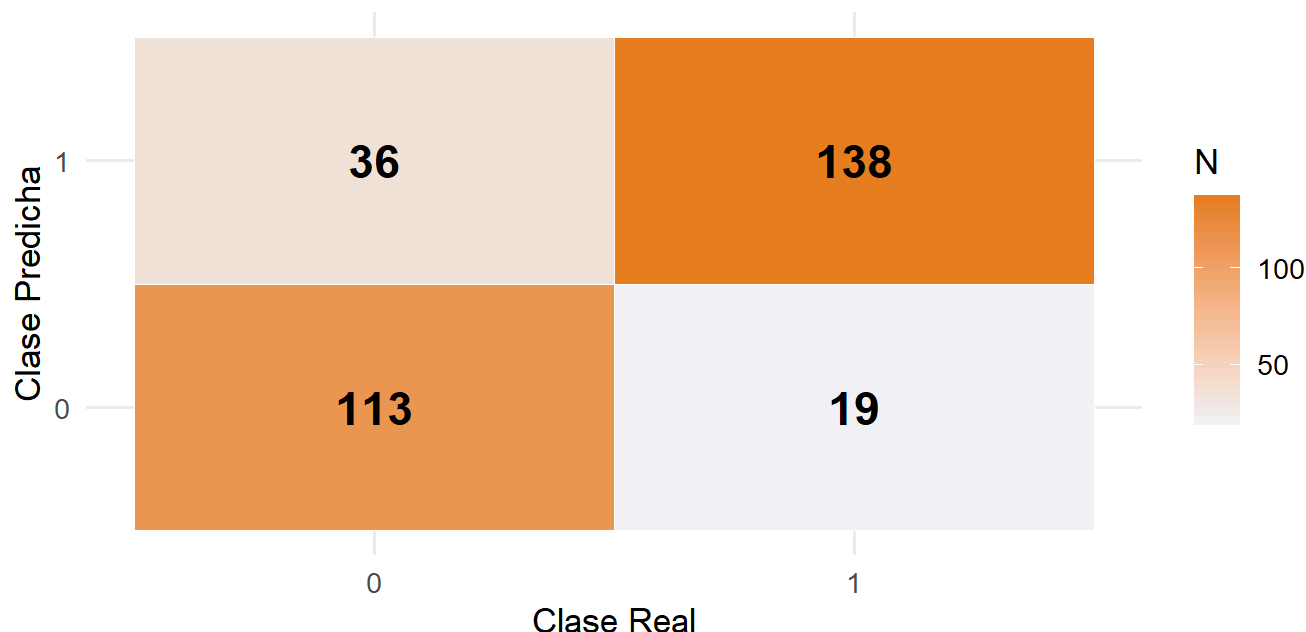
as.data.frame(cfm_lp$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 6, fontface = "bold") +
  scale_fill_gradient(low = "#f0f4f8", high = "#e67e22") +
  labs(
    title = "Matriz de Confusión - Label Propagation (k = 15, 10% etiquetado)",
    subtitle = paste0("Accuracy: ", round(cfm_lp$overall["Accuracy"] * 100, 1), "% | F1: ",

```

```
round(cfm_lp$byClass["F1"], 3)),  
  x = "Clase Real", y = "Clase Predicha", fill = "N"  
) +  
theme_minimal(base_size = 13)
```

Matriz de Confusión – Label Propagation (k = 15, 10% etiquetado)

Accuracy: 82% | F1: 0.834



Label Propagation también alcanzó un accuracy del 82% y un F1 de 0.834, igualando al supervisado en F1 a pesar de usar los mismos datos etiquetados. Lo que distingue a este modelo es su comportamiento: detectó 138 pacientes con enfermedad correctamente, el número más alto de los tres, pero a cambio cometió 36 falsos positivos, también el número más alto. En la práctica, esto significa que el modelo tiende a ser más agresivo clasificando personas como enfermas. Para diagnóstico cardíaco eso no es necesariamente malo: es preferible enviar a un paciente sano a un examen adicional que dejar pasar una enfermedad real. Los 19 falsos negativos son los más bajos de los tres modelos, lo que lo convierte en el más sensible.

En el contexto médico de este dataset, los falsos negativos (paciente con enfermedad clasificado como sano) son el error más costoso. Los tres modelos con 10% de etiquetas generaron 29, 30 y 19 falsos negativos respectivamente (Supervisado, Self-Training, Label Propagation). Self-Training y Label Propagation logran mantener este error comparable al modelo supervisado puro, validando su utilidad en escenarios donde el etiquetado es escaso.

Predicción vs Valor Real

```

comparacion <- data.frame(
  Indice    = 1:nrow(test_data),
  Real      = as.integer(as.character(test_data$target)),
  Pred_ST   = as.integer(as.character(pred_st_vec)),
  Pred_SUP  = as.integer(as.character(pred_sup_10)),
  Pred_LP   = as.integer(as.character(pred_lp_vec))
) %>%
  mutate(
    Error_ST = Real != Pred_ST,
    Error_SUP = Real != Pred_SUP,
    Error_LP  = Real != Pred_LP
  )

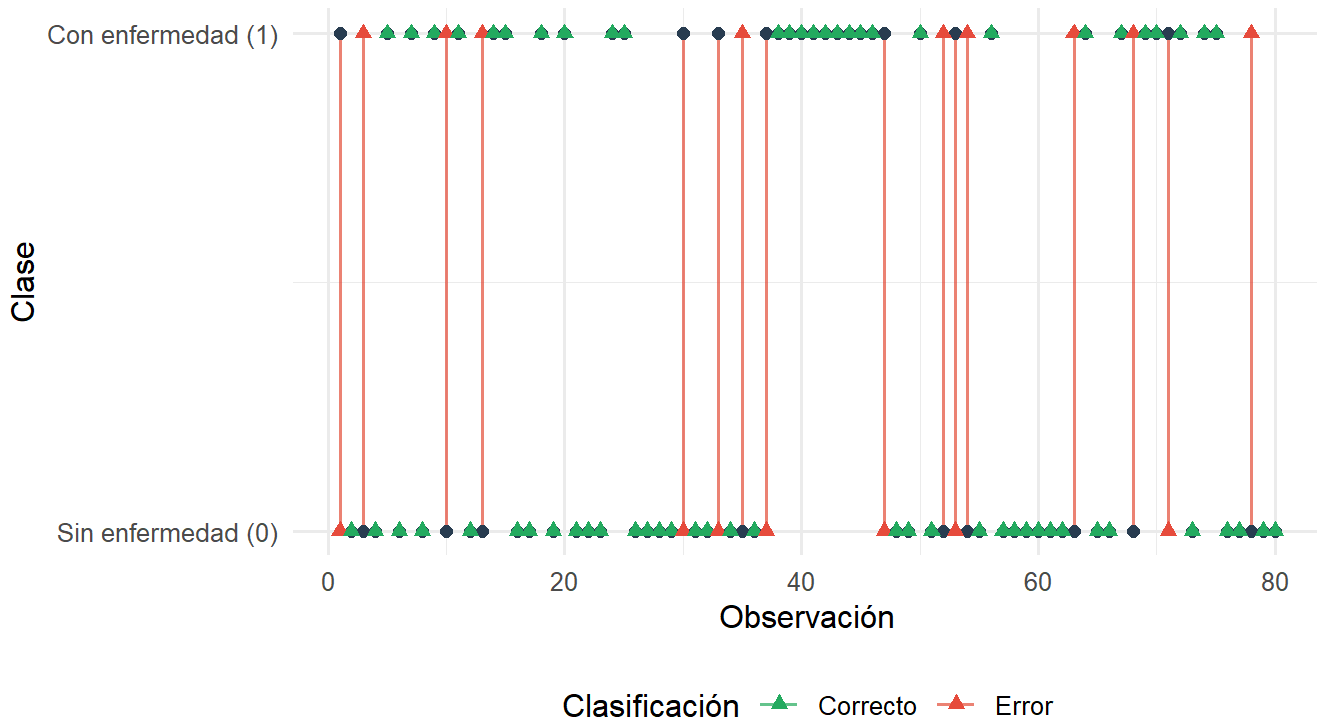
set.seed(42)
muestra_idx <- sort(sample(1:nrow(comparacion), 80))
comp_muestra <- comparacion[muestra_idx, ] %>%
  mutate(Indice = row_number())

ggplot(comp_muestra) +
  geom_segment(aes(x = Indice, xend = Indice, y = Real, yend = Pred_ST,
                  color = Error_ST), linewidth = 0.6, alpha = 0.7) +
  geom_point(aes(x = Indice, y = Real), shape = 16, size = 2, color = "#2c3e50") +
  geom_point(aes(x = Indice, y = Pred_ST, color = Error_ST), shape = 17, size = 2) +
  scale_color_manual(values = c("FALSE" = "#27ae60", "TRUE" = "#e74c3c"),
                    labels = c("Correcto", "Error")) +
  scale_y_continuous(breaks = c(0, 1), labels = c("Sin enfermedad (0)", "Con enfermedad (1)")) +
  labs(
    title    = "Predicción vs Valor Real - Self-Training (muestra de 80 observaciones)",
    subtitle = "Círculos = valor real | Triángulos = predicción | Línea roja = error",
    x        = "Observación", y = "Clase", color = "Clasificación"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")

```


Predicción vs Valor Real – Self-Training (muestra de 80 obser

Círculos = valor real | Triángulos = predicción | Línea roja = error



```
errores_resumen <- data.frame(
  Modelo      = c("Supervisado (RF)", "Self-Training", "Label Propagation"),
  Correctos   = c(sum(!comparacion$Error_SUP),
                  sum(!comparacion$Error_ST),
                  sum(!comparacion$Error_LP)),
  Errores     = c(sum(comparacion$Error_SUP),
                  sum(comparacion$Error_ST),
                  sum(comparacion$Error_LP))
) %>%
  pivot_longer(c(Correctos, Errores), names_to = "Tipo", values_to = "N")

ggplot(errores_resumen, aes(x = Modelo, y = N, fill = Tipo)) +
  geom_col(position = "stack", width = 0.5) +
  geom_text(aes(label = N), position = position_stack(vjust = 0.5),
            size = 4, fontface = "bold", color = "white") +
  scale_fill_manual(values = c("Correctos" = "#27ae60", "Errores" = "#e74c3c")) +
  labs(
    title = "Predicciones correctas vs errores por modelo (10% etiquetado, n = 306)",
    x = "", y = "Número de observaciones", fill = ""
  ) +
  theme_minimal(base_size = 13) +
  theme(legend.position = "bottom")
```

Predicciones correctas vs errores por modelo (10% etiquetado, n :

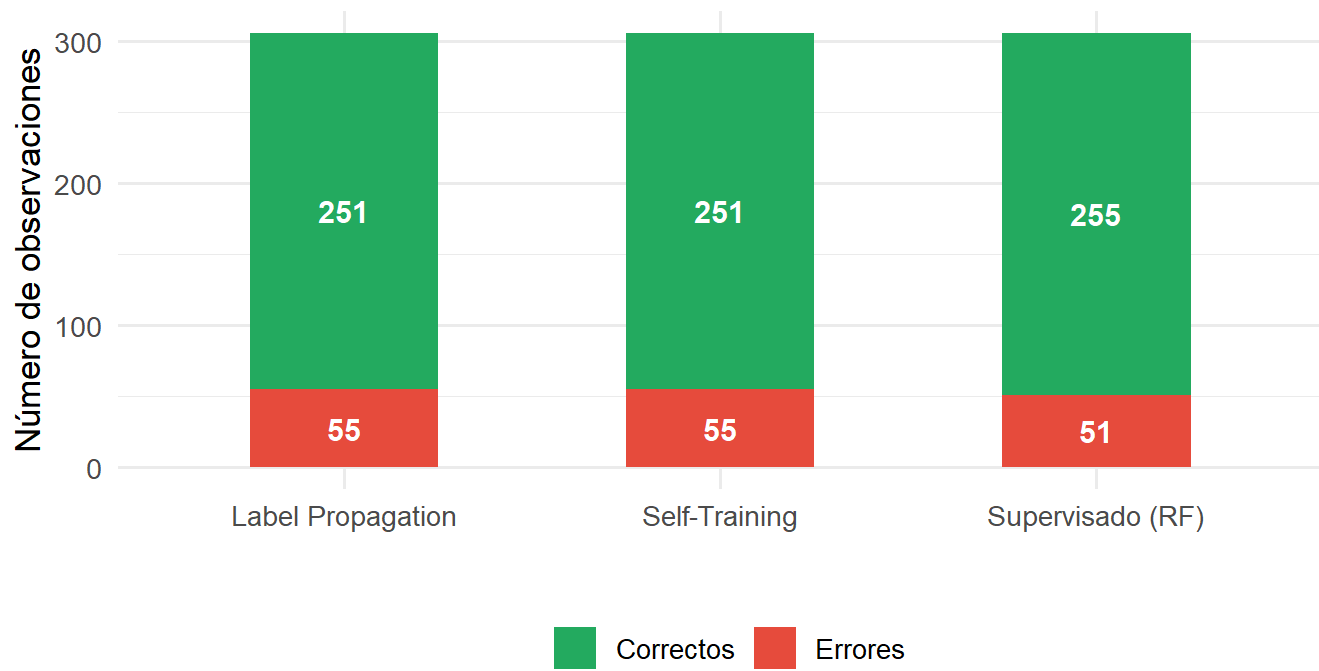


Tabla Resumen Final

```

resumen_final <- data.frame(
  Modelo = c(
    "Supervisado (RF)",
    "Supervisado (RF)",
    "Supervisado (RF)",
    "Self-Training (thr=0.95)",
    "Self-Training (thr=0.95)",
    "Self-Training (thr=0.90)",
    "Label Propagation (k=15)",
    "Label Propagation (k=15)",
    "Label Propagation (k=10)"
  ),
  Etiquetado = c("5%", "10%", "20%", "5%", "10%", "20%", "5%", "10%", "20%"),
  Accuracy = c(
    resultados$accuracy[resultados$modelo == "Supervisado - Random Forest" & resultados$porcenta
je_etiquetado == "5%"],
    resultados$accuracy[resultados$modelo == "Supervisado - Random Forest" & resultados$porcenta
je_etiquetado == "10%"],
    resultados$accuracy[resultados$modelo == "Supervisado - Random Forest" & resultados$porcenta
je_etiquetado == "20%"],
    mejor_threshold$accuracy[mejor_threshold$porcentaje_etiquetado == "5%"],
    mejor_threshold$accuracy[mejor_threshold$porcentaje_etiquetado == "10%"],
    mejor_threshold$accuracy[mejor_threshold$porcentaje_etiquetado == "20%"],
    mejor_k$accuracy[mejor_k$porcentaje_etiquetado == "5%"],
    mejor_k$accuracy[mejor_k$porcentaje_etiquetado == "10%"],
    mejor_k$accuracy[mejor_k$porcentaje_etiquetado == "20%"]
  ),
  F1 = c(
    resultados$f1[resultados$modelo == "Supervisado - Random Forest" & resultados$porcentaje_eti
quetado == "5%"],
    resultados$f1[resultados$modelo == "Supervisado - Random Forest" & resultados$porcentaje_eti
quetado == "10%"],
    resultados$f1[resultados$modelo == "Supervisado - Random Forest" & resultados$porcentaje_eti
quetado == "20%"],
    mejor_threshold$f1[mejor_threshold$porcentaje_etiquetado == "5%"],
    mejor_threshold$f1[mejor_threshold$porcentaje_etiquetado == "10%"],
    mejor_threshold$f1[mejor_threshold$porcentaje_etiquetado == "20%"],
    mejor_k$f1[mejor_k$porcentaje_etiquetado == "5%"],
    mejor_k$f1[mejor_k$porcentaje_etiquetado == "10%"],
    mejor_k$f1[mejor_k$porcentaje_etiquetado == "20%"]
  )
)

kable(resumen_final, digits = 4,
      col.names = c("Modelo", "% Etiquetado", "Accuracy", "F1-score"),
      caption = "Tabla resumen - mejores configuraciones por modelo y % de etiquetado") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"),
                full_width = FALSE) %>%

```

```
row_spec(which(resumen_final$F1 == max(resumen_final$F1)),
         bold = TRUE, color = "white", background = "#27ae60")
```

Tabla resumen – mejores configuraciones por modelo y % de etiquetado

Modelo	% Etiquetado	Accuracy	F1-score
Supervisado (RF)	5%	0.7876	0.7910
Supervisado (RF)	10%	0.8497	0.8506
Supervisado (RF)	20%	0.8791	0.8825
Self-Training (thr=0.95)	5%	0.8072	0.8078
Self-Training (thr=0.95)	10%	0.8333	0.8339
Self-Training (thr=0.90)	20%	0.8824	0.8889
Label Propagation (k=15)	5%	0.7941	0.8131
Label Propagation (k=15)	10%	0.8203	0.8338
Label Propagation (k=10)	20%	0.8693	0.8773

Conclusiones

Después de haber evaluado los tres modelos con solo el 10% de datos etiquetados, lo que más sorprende no es cuál ganó, sino cuánto se acercaron todos al mismo resultado. Partiendo de apenas 72 observaciones etiquetadas sobre más de 700 disponibles para entrenamiento, los dos algoritmos semi-supervisados lograron rendimientos prácticamente indistinguibles del modelo supervisado. Eso, en sí mismo, es el hallazgo más importante. Si tuviéramos que elegir uno para implementar, nos inclinaríamos por Label Propagation con $k = 15$. No porque tenga el accuracy más alto, sino porque en este problema concreto, el cual es detectar una enfermedad cardíaca, el tipo de error importa. Este modelo fue el que menos pacientes enfermos dejó sin detectar, y eso tiene un peso real. Que haya clasificado más falsos positivos es un costo aceptable, lo que significa más revisiones innecesarias, pero no pacientes enviados a casa sin diagnóstico. Self-Training es la opción más predecible y estable. Si el objetivo fuera desplegar el modelo en un entorno donde el comportamiento consistente importa más que maximizar detección, sería la elección. Responde bien a cambios de hiperparámetros y su lógica de pseudo-etiquetado es fácil de auditar y explicar. Lo que este experimento deja claro es que el aprendizaje semi-supervisado no es un truco ni un atajo, es una solución real para escenarios donde etiquetar datos cuesta dinero, tiempo o expertise. En medicina, eso es casi siempre el caso. Con el 20% de datos etiquetados, los modelos ya superan el 88% de accuracy, lo que sugiere que en la práctica no haría falta etiquetar más allá de ese umbral para obtener un modelo útil.